

BENUTZERHANDBUCH

Maprisma

& AI Agents Map

Deine KI-Agenten, lokalen Modelle, MCP-Server und Skills auf einer Karte

Maprisma 0.12.0 · AI Agents Map 0.26.1

Oktober 2026

Über dieses Handbuch

Dieses Handbuch beschreibt eine Anwendung, die es in zwei Ausgaben gibt:

Ausgabe	Maprisma	AI Agents Map
Was es ist	Eine eigenständige GTK 4-/libadwaita-App für jeden Linux-Desktop	Ein Plugin für den Omarchy-Desktop, geöffnet über einen Knopf in der Leiste
Installation aus	AUR (`maprisma-bin`) oder AppImage	Dem Omarchy Plugin Marketplace
Aussehen	Folgt dem hellen/dunklen Modus und der Akzentfarbe des Systems	Farben des Omarchy-Themes oder GNOMEs Adwaita-Aussehen
Einstellungsordner	`~/.config/maprisma/`	`~/.config/omarchy/ai-agents-map/`

Alles andere — Setup, die Karte, Verbinden durch Zeichnen, lokale LLMs, Modelle, MCP-Server und Skills — ist in beiden gleich. Was nur für eine Ausgabe gilt, ist mit MAPRISMA oder OMARCHY markiert.

Schreibweisen

Hinzufügen	Ein Knopf, Reiter oder Menüeintrag im Fenster.
Einstellungen › **Aussehen**	Ein Weg durch die Oberfläche: Einstellungen öffnen, dann **Aussehen**.
`~/.agents/skills`	Eine Datei, ein Ordner oder ein Befehl. `~` ist dein persönlicher Ordner.
`Strg`+`,`	Ein Tastenkürzel.

> **HINWEIS** Hinweise geben nützlichen Hintergrund.

> **TIPP** Tipps zeigen einen schnelleren Weg oder eine gute Gewohnheit.

> **VORSICHT** Vorsicht-Kästen weisen auf etwas hin, das schwer rückgängig zu machen ist.

Inhalt

1 Einführung 1
1.1 Was es kann 1
1.2 Grundbegriffe 2
1.3 Nichts ändert sich ohne Vorschau 2
2 Installation 3
2.1 Maprisma installieren 3
2.2 Erster Start 3
2.3 Die App entfernen 4
3 Das Fenster im Überblick 5
3.1 Setup 6
3.2 Auswählen, was die Karte zeigt 8
3.3 Mit der Karte arbeiten 9
3.4 Offene Probleme 10
3.5 Offene Probleme in der Flutter-Vorschau 11
3.6 Updates und Vorschläge 11
3.7 Add-on-Vorschläge in der Flutter-Vorschau 11
4 Agenten 12
4.1 Details eines Agenten 12
4.2 Agenten starten und stoppen 13
4.3 Netzwerkverkehr 14
4.4 Subagenten 15
4.5 Agenten, die zusammenarbeiten, in der Flutter-Vorschau 15
4.6 Kosten in der Flutter-Vorschau 15
4.7 Details in der Flutter-Vorschau 16
4.8 Terminal-Tabs, Nutzungsbox und Startmodell in der Flutter-Vorschau 16
4.9 Einrichten… 16
4.10 Anmelden 16
4.11 API-Schlüssel in der Flutter-Vorschau 17
4.12 Agenten aktualisieren 18
4.13 Einen laufenden Agenten in der Flutter-Vorschau aktualisieren 18
4.14 Agenten, die die App installieren kann 18
4.15 Neue Agenten 18
5 Lokale LLMs 19
5.1 Ein lokales LLM installieren 19
5.2 Installieren in der Flutter-Vorschau 19
5.3 Mit den Agenten starten und stoppen 20
5.4 Details eines lokalen LLMs 20
5.5 Leistung 20
5.6 Leistung in der Flutter-Vorschau 21
5.7 GPU-Speicher und GPU-Speicher freigeben 21
5.8 Modelle gleichzeitig 22
5.9 Modelle gleichzeitig in der Flutter-Vorschau 22
5.10 Von anderen Apps genutzt 22
6 Modelle 24
6.1 Von Hugging Face holen 24
6.2 Unterbrochene Downloads in der Flutter-Vorschau 25
6.3 Einen Agenten fragen, welches Modell passt 25
6.4 Installiert 25
6.5 Kontextgröße 26
6.6 Auf diesem PC 26
6.7 Modelle verschieben und löschen 26
6.8 Ein Modellordner 26
7 Verbinden durch Zeichnen 27
7.1 Agent ↔ lokales LLM 27
7.2 Agent ↔ MCP-Server 27
7.3 Agent ↔ Skill-Ordner 27
7.4 Auf diesem PC gefunden 27
7.5 Lokale Verbindungen in der Flutter-Vorschau 28
7.6 Auf diesem PC gefunden in der Flutter-Vorschau 28
7.7 Wenn eine Verbindung fehlschlägt 29
8 MCP-Server 30
8.1 Woher Server kommen 30
8.2 Einen Server einrichten 31
8.3 MCP-Server in der Flutter-Vorschau 31
8.4 Server verwalten 31
9 Skills 33
9.1 Skills finden 33
9.2 Skills aus dem Web in der Flutter-Vorschau 33
9.3 Skill-Ordner 33
10 Einstellungen und Hilfe 34
10.1 Flutter-Design und Kartenlichter 34
10.2 Das Popup unter der Leiste auf Omarchy 34
10.3 Nutzungszahlen und Ecken von Omarchy in der Flutter-Vorschau 34
10.4 Einstellungen 35
10.5 Rote Limits in der Flutter-Vorschau 36
10.6 Letzte Änderungen und Rückgängig nach einem Neustart 37
10.7 Hilfe 37
10.8 Hilfe in der Flutter-Vorschau 37
10.9 Speicherplatz 37
11 Smart Check 39
11.1 Zusammenfassung des PC-Setups 40
11.2 Gefundene Updates 40

11.3 Während ein Agent arbeitet 41
11.4 Der lokale Helfer 41
11.5 KI-Prüfungen und der lokale Helfer in der Flutter-Vorschau 41
11.6 Protokoll 42
11.7 Token-Statistik 42
12 Gruppenkarten und Add-ons auf der Karte 43
12.1 Eine Kategorie erweitern 43
12.2 Add-ons verwalten 43
12.3 Erster Start und vorhandene Abläufe 43
13 Die Flutter-KI-Karte bearbeiten 45
14 Aufgabenkarten auf der Karte 46
15 Systemkarte 47
15.1 Erster Start und separate Datenträger 47
15.2 Gespeicherte Karte und Aktualisierung 47
15.3 Verbinden, Verschieben und Rückgängig 47
15.4 Analyse und KI-Hilfe 48
16 Gesundheits- und Sicherheitscheck 49
16.1 Die Karten lesen 49
16.2 Eine Einstellung ändern 50
16.3 Sicherheit in der Flutter-Vorschau 50
16.4 Zustand in der Flutter-Vorschau 51
16.5 Was bei jedem Agenten geprüft wird 51
16.6 Das Risiko steuern 51
16.7 Die Begriffe erklärt 52
17 Add-ons 54
17.1 Maprisma einrichten 54
17.2 Die Add-on-Seite 55
17.3 Installieren, ein- und ausschalten, aktualisieren, entfernen 56
17.4 Datenschutzstufen 57
17.5 Arten von Add-ons 57
17.6 Ein neuer Agent auf deinem Computer 58
17.7 Ein Add-on erstellen 58
18 Datenschutz und Sicherheit 59
19 Problemlösung 60
A Referenz 61
A.1 Tastenkürzel 61
A.2 Bedienung per Tastatur in der Flutter-Vorschau 61
A.3 Dateien und Ordner 61
A.4 Glossar 62

KAPITEL 1

Einführung

Maprisma zeigt das KI-Setup deines Computers als Karten und als Karte: welche Coding-Agenten installiert sind, welche Sprachmodelle sie nutzen — Cloud-Dienste und lokale Laufzeiten —, welche Modelle diese bereitstellen und welche MCP-Server und Skills jeder Agent hat. Und es lässt dich dieses Setup durch Linienzeichnen ändern.

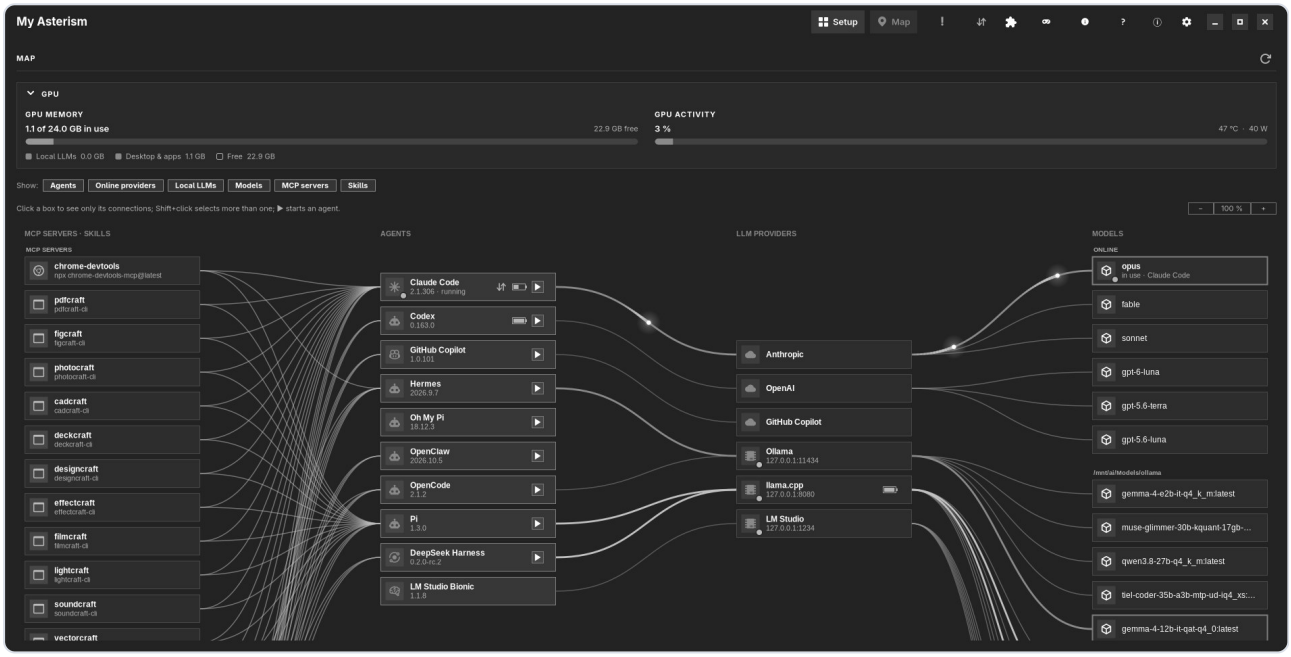


Abbildung 1 · Das Hauptfenster: die Karte über die ganze Breite, darüber die GPU-Karte, in der Kopfzeile das ! für offene Probleme.

1.1 Was es kann

- **Coding-Agenten** — Claude Code, Codex, GitHub Copilot, Cursor CLI, Gemini, OpenCode, Pi, Hermes und mehr: ihre Version, ihr Abo, ihre Limits und ihre Nutzung, und was jedem noch fehlt, mit dem Befehl, der es behebt.
- **Lokale LLMs** — llama.cpp, Ollama, LM Studio, vLLM, Jan und KoboldCpp: installieren, Modelle herunterladen, die zur Grafikkarte passende Kontextgröße einstellen und sie zusammen mit den Agenten starten, die sie nutzen.
- **MCP-Server und Skills** — gefunden auf deinem Computer oder im Web, mit empfohlenen Servern und einer Suche in der offiziellen MCP-Registry.
- **Offene Probleme** — ein ! in der Kopfzeile, wenn etwas zu beheben ist, jedes Problem mit dem Weg dorthin, wo es behoben wird, und wofür dein Setup taugt.

1.2 Grundbegriffe

Agent	Ein Programm, mit dem du sprichst und das Code schreibt oder Aufgaben erledigt, etwa Claude Code.
LLM-Anbieter	Wo das Modell eines Agenten läuft: ein Cloud-Dienst (Anthropic, OpenAI, …) oder eine lokale LLM-Laufzeit auf diesem Computer (llama.cpp, Ollama, …).
Modell	Das Sprachmodell selbst. Lokale Modelle sind Dateien, meist im GGUF-Format.
MCP-Server	Ein kleines Programm oder ein Webdienst, der Agenten zusätzliche Werkzeuge gibt — Dateien lesen, im Web suchen, einen Browser steuern. MCP ist das Model Context Protocol.
Skill	Ein Ordner mit Anweisungen (einer `SKILL.md`) und manchmal Skripten, die einem Agenten eine bestimmte Aufgabe beibringen.

1.3 Nichts ändert sich ohne Vorschau

Jede Änderung an den Einstellungen eines Agenten wird zuerst Zeile für Zeile gezeigt, bevor etwas geschrieben wird. Die alte Datei bleibt als Sicherung erhalten, und die Leiste, die nach der Änderung erscheint, hat **Rückgängig**. Die App selbst läuft nie mit Administratorrechten; nur das Installieren eines Agenten, der als Systempaket kommt, fragt nach deinem Passwort, in einem Terminal, das du siehst.

KAPITEL 2

Installation

2.1 Maprisma installieren

Maprisma ist ein einziges Programm für Linux auf x86-64-Rechnern. Es braucht kein Python und bringt seine Symbole und seine signierten Add-ons selbst mit. Wähle einen dieser Wege:

- **Arch Linux und Omarchy (AUR):** Installiere das Paket `maprisma-bin` mit deinem AUR-Helfer, zum Beispiel `yay -S maprisma-bin`. Es legt die App nach `/usr/lib/maprisma`, den Befehl `maprisma` in deinen Pfad und Maprisma in den App-Starter.
- **AppImage, für andere Distributionen:** Lade `Maprisma-<Version>-x86_64.AppImage` von `github.com/maprisma/maprisma/releases`, mach es ausführbar (`chmod +x Maprisma-*.AppImage`) und starte es. Es nutzt das GTK 3, das fast jeder Linux-Desktop hat.

> **HINWEIS** Das AppImage läuft aus einem temporären Ordner und richtet deshalb den Hintergrunddienst für Einstellungen › **Lokale LLMs mit ihren Agenten starten** nicht ein. Nimm das AUR-Paket, wenn lokale LLMs mit ihren Agenten starten und stoppen sollen.

Optionale Programme: `xdg-terminal-exec` startet Agenten in deinem eigenen Terminal, `xdg-utils` öffnet das Handbuch und Webseiten, und `mise` installiert und aktualisiert Agenten.

Maprisma speichert seine Einstellungen in `~/.config/maprisma`; ein absolutes `XDG_CONFIG_HOME` verlegt diesen Ordner nach `$XDG_CONFIG_HOME/maprisma`. Auf Omarchy nutzt es die Desktop-Integration von selbst: `maprisma --toggle` öffnet das Fenster als Popup unter der Leiste, holt es nach vorn oder blendet es aus.

2.2 Erster Start

Paket und AppImage brauchen keine weitere Einrichtung. Wenn du Maprisma unter seinem früheren Namen Asterism benutzt hast, kopiert der erste Start einmalig die Ordner von Asterism; die Originale bleiben unverändert.

Das Fenster ist ein normales App-Fenster: Es kann gekachelt, schwebend, auf einen anderen Arbeitsbereich verschoben und mit `Alt`+`Tab` erreicht werden. MAPRISMA Ein neues Maprisma öffnet sich auf seiner **Willkommensseite**: Die auf diesem Computer installierten Agenten und lokalen LLMs sind Karten, die schon angehakt sind, und **Einschalten** bringt sie auf die Karte (siehe Maprisma einrichten). OMARCHY Der Leisten-Knopf öffnet das Fenster, holt es nach vorn oder versteckt es, wenn es schon vorn ist. Schließen versteckt das Fenster; das Programm läuft im Hintergrund weiter, damit es sofort wieder aufgeht.

> **HINWEIS** Läuft ein Agent im Fenster, fragt das Schließen zuerst: **Stoppen und schließen** (dann stoppt auch das lokale LLM des Agenten und gibt den Grafikspeicher frei) oder **Weiterlaufen lassen**.

Während die App beim ersten Öffnen schaut, was auf diesem Computer ist, zeigt eine animierte Einblendung, was sie findet, Art für Art: Agenten, lokale LLMs, MCP-Server und Skills, jeweils mit einem Kreisel, der zu ✓ und einer Zahl wird, und wie viele davon noch nicht verbunden sind. Sie blendet sich nach einem Moment von selbst aus; ein Klick schließt sie sofort. Das Fenster öffnet sich sofort, mit dieser Übersicht als Kasten in seiner Mitte; dahinter füllt sich die Karte.

2.3 Die App entfernen

AUR: `sudo pacman -R maprisma-bin`. AppImage: die Datei löschen.

Schalte vor dem Entfernen Einstellungen › **Lokale LLMs mit ihren Agenten starten** aus, damit auch der Hintergrund-Wächter entfernt wird. Was du mit der App geändert hast, bleibt: Einstellungen der Agenten, MCP-Server, Skills und installierte lokale LLMs. Deinstalliere ein lokales LLM vorher (Rechtsklick › **Deinstallieren…**), wenn es weg soll. Um auch den eigenen Zustand der App zu löschen:

```
rm -rf ~/.config/maprisma ~/.cache/maprisma \
       ~/.local/share/maprisma ~/.local/state/maprisma
```

KAPITEL 3

Das Fenster im Überblick

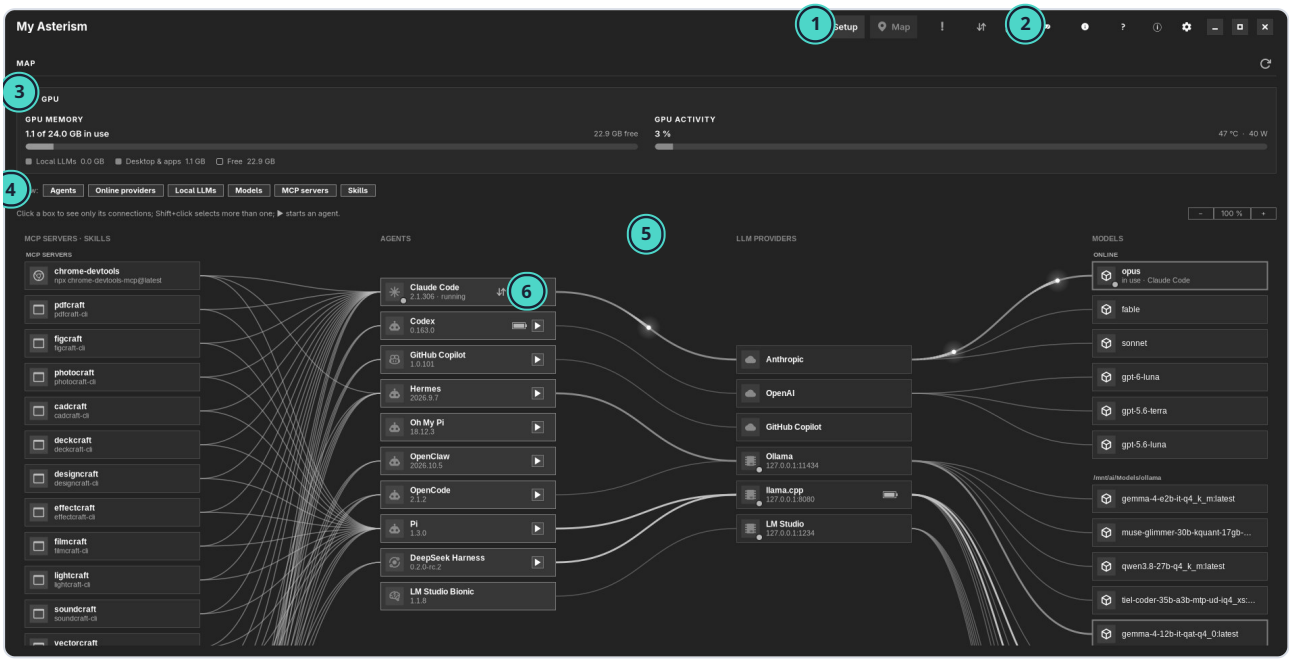


Abbildung 2 · Das Hauptfenster. Die Nummern entsprechen der Liste darunter.

1. **Setup** und **Karte** — die beiden Ansichten, auf jeder Seite; die gerade sichtbare ist hervorgehoben. Setup zeigt jeden Agenten, jedes lokale LLM, jeden MCP-Server, jeden Skill-Ordner und jedes Modell als Karte: installieren, aktualisieren, anmelden, entfernen (siehe unten).
2. Die Symbole der Kopfzeile: ↻ aktualisieren (F5: erneut nachsehen, wie beim Start), **!** offene Probleme und Updates (rot: etwas zu beheben; Akzentfarbe: nur Updates und Vorschläge; nicht da, wenn es nichts gibt), ⓘ die Legende (was jeder Punkt, jede Linie und jede Markierung der Karte bedeutet), ⇅ der Netzwerkverkehr der laufenden Agenten (siehe Netzwerkverkehr), die zwei Funkelsterne für **Smart Check** (nur solange Claude Code oder Codex installiert ist, siehe Smart Check), die Add-ons MAPRISMA, das gefüllte **i** für Information (siehe Hilfe), **?** Hilfe (F1) und das Zahnrad für die Einstellungen (Strg + ,). Halte die linke Maustaste auf einem Symbol gedrückt und zieh es zur Seite, um es woanders hinzusetzen: Ein Balken zeigt, wo es landet, und die Reihenfolge bleibt erhalten.
3. **GPU** — GPU-Speicher und GPU-Aktivität: wie die Grafikkarte gerade genutzt wird, in einer Karte, die sich zuklappt (zugeklappt behält ihre Zeile die Zahlen).
4. **Zeigen**-Schalter — kürzen die Karte auf die Bereiche, an denen du arbeitest.
5. **Die Karte** — MCP-Server und Skills ← Agenten → LLM-Anbieter → Modelle, über die ganze Breite, unter ihrer Überschrift **KARTE**.
6. ▶ im Feld eines Agenten startet ihn in einem Terminal im Fenster; während er dort läuft, holt der Knopf sein Terminal zurück.

3.1 Setup

Die Karte zeigt, was eingerichtet ist — installierte Agenten und lokale LLMs, die MCP-Server und Skill-Ordner, die Agenten nutzen, und Modelle — und, blass, die auf diesem Computer gefundenen MCP-Server und Skill-Ordner, die noch kein Agent nutzt, bereit zum Verbinden mit einer Linie (siehe Auf diesem PC gefunden). Alles andere geschieht im **Setup** (die Schaltfläche in der Kopfzeile neben **Karte**), eine Karte pro Eintrag, in Abschnitten, die sich mit einem Klick auf ihre Leiste einklappen:

- **Wie etwas auf deine Karte kommt** — eine zugeklappte Karte ganz oben; öffne sie für fünf nummerierte Schritte: einschalten (oder finden), was du nutzt, installieren, was fehlt, bei einem Online-Agenten anmelden, durch Zeichnen verbinden, und was von selbst erscheint.
- **Was du damit machen kannst** — wofür das Setup taugt (Alltagshilfe, Programmieren, Bilder, Sprache, Recherche und Schreiben), je eine Kachel mit dem, was da ist, und dem nächsten Schritt; zugeklappt sagt ihre Zeile, wie viele bereit sind.
- **KI-Agenten, Lokale LLMs** — alle, die die App kennt. Eine Karte mit blauem Rahmen ist auf diesem PC installiert; eine graue nicht, und sie hat **Installieren**, wo installierte **Deinstallieren** haben. OMARCHY Ein Klick auf eine installierte Karte öffnet ihre Details. MAPRISMA Ein Klick hakt die Karte an oder ab (✓), dann schaltet **Übernehmen** ihr Add-on ein oder aus. Ausschalten nimmt sie von der Karte; gelöscht wird nichts.
- **Online-Anbieter** MAPRISMA — das Add-on der Online-Anbieter; sein Menü hat **Anmelden bei …** für jeden installierten Agenten, der sie nutzt.
- **MCP-Server, Skills, Modelle** — eine Karte für jeden installierten Eintrag. Ein Klick öffnet seine Details (bei einem Modell sein Menü). Die Karte mit **+** am Ende eines Abschnitts fügt einen hinzu: **MCP-Server hinzufügen**, **Skills hinzufügen**, **Modell hinzufügen**, **Add-ons holen**.
- ··· oder ein Rechtsklick auf eine Karte öffnet dasselbe Menü wie das Feld auf der Karte: aktualisieren, anmelden, starten oder stoppen, Details, deinstallieren oder entfernen, ein Modell verschieben.
- Ein gelbes ↑ **Version** auf einer Karte heißt, dass ein Update bereitsteht; ihr Menü hat **Aktualisieren**.
- Ein rotes **!** auf einer Karte heißt: Dort gibt es ein offenes Problem; zeige mit der Maus darauf, um zu sehen, welches.
- Eine kleine Batterie auf der Karte eines Online-Agenten zeigt, was von seinen Limits übrig ist (eine Sitzung, eine Woche): Das knappste zählt, gelb bei einem Viertel oder weniger, rot unter einem Zehntel; zeige mit der Maus darauf für jedes Limit und wann es zurückgesetzt wird. Auf der Karte eines laufenden lokalen LLMs ist die Batterie der noch freie GPU-Speicher, in denselben Farben; zeige mit der Maus darauf, um zu sehen, wie viel dieses lokale LLM belegt.
- Das Aktualisieren-Symbol (F5) in der Kopfzeile sieht erneut nach, was installiert, eingerichtet und angemeldet ist, und führt die Prüfungen eines Starts noch einmal aus (neue Apps, Add-ons, Updates, offene Probleme).

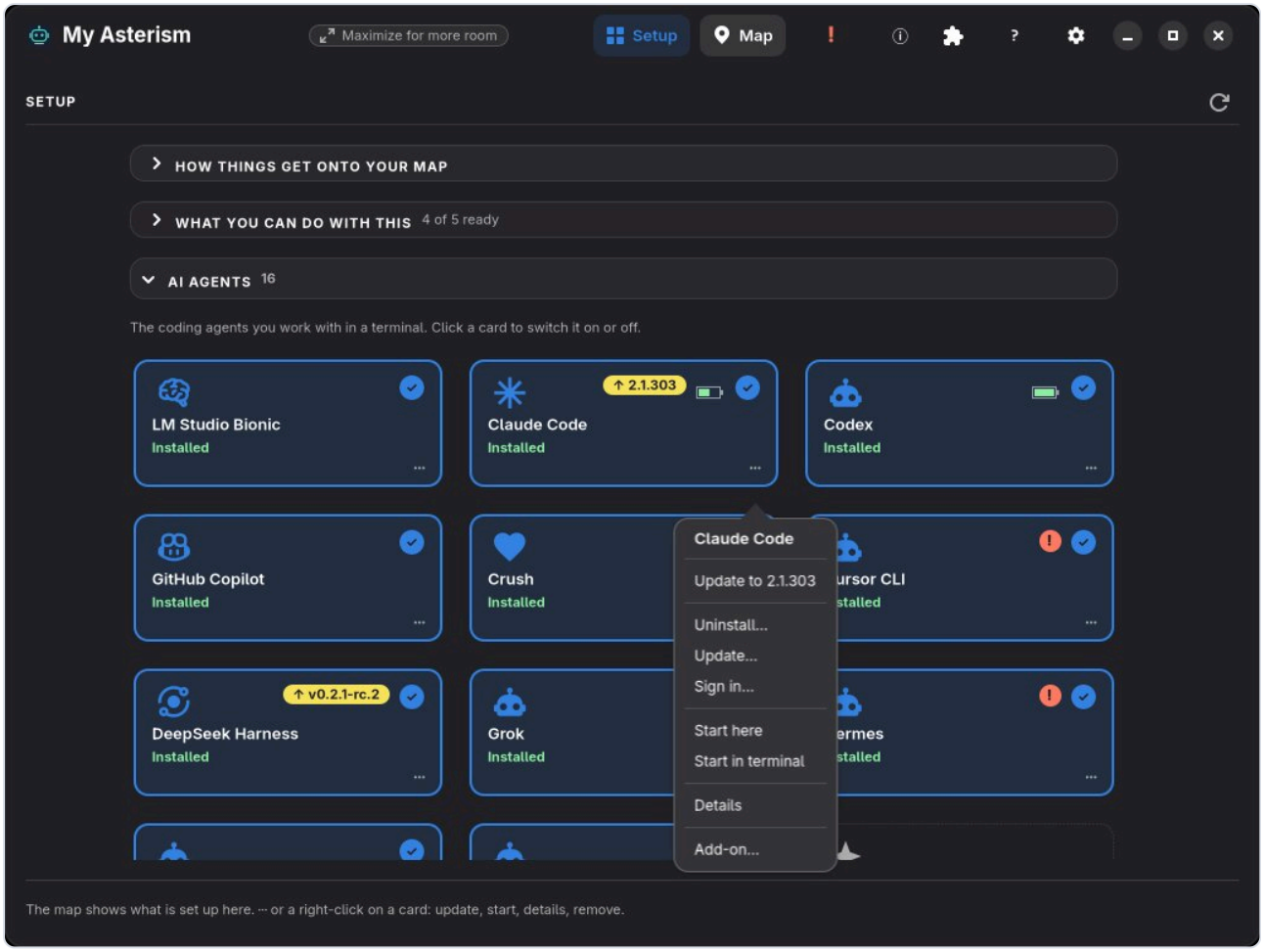


Abbildung 3 · Setup: Installierte Agenten haben einen blauen Rahmen, eingeschaltete ein ✓, Updates ein gelbes und Probleme ein rotes Abzeichen; das Menü von Claude Code.

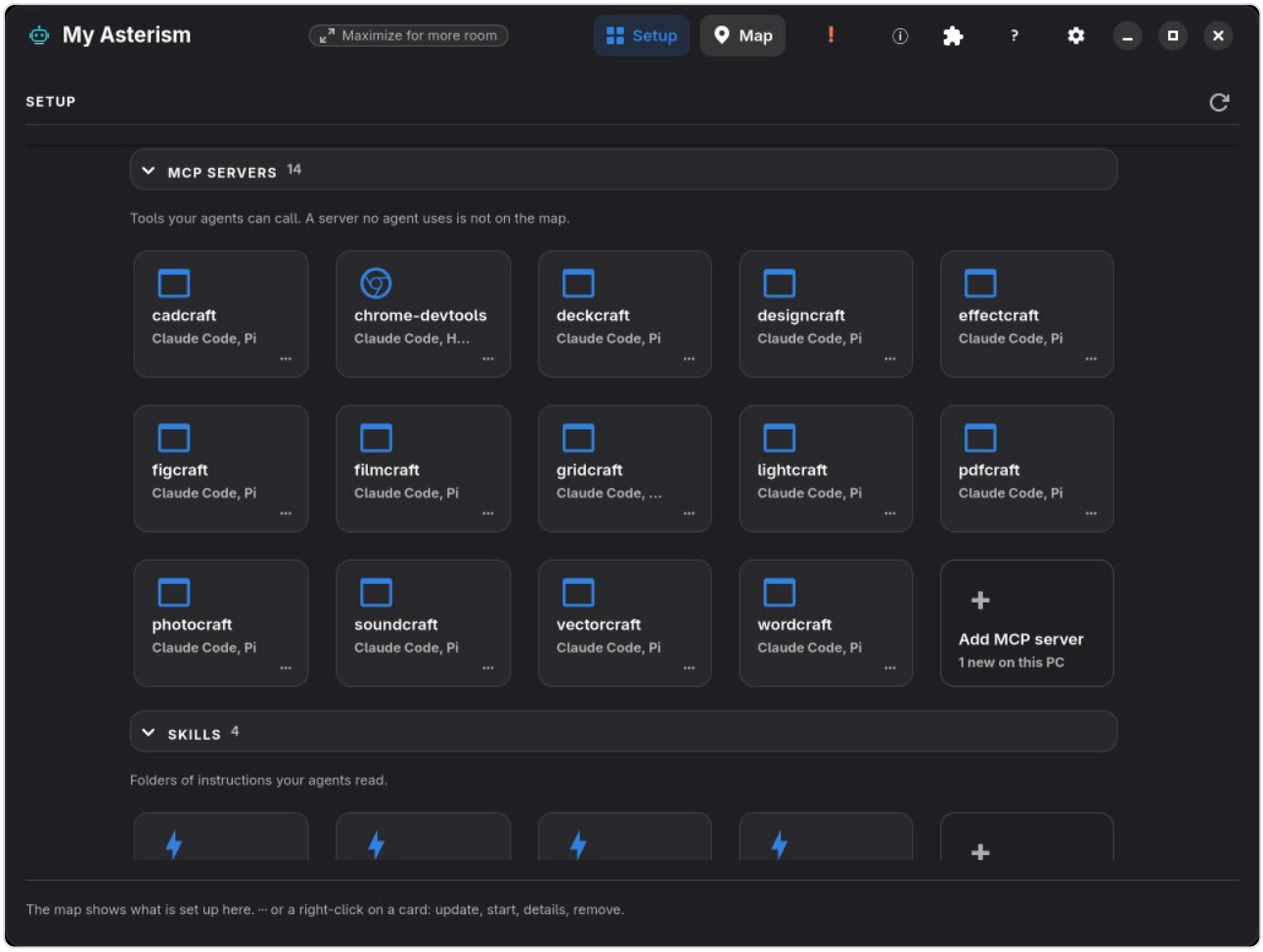


Abbildung 4 · MCP-Server und Skills, jeder Abschnitt mit einer Karte zum Hinzufügen.

Es gibt nichts zu verstecken: Was du nicht auf der Karte willst, wird hier ausgeschaltet oder entfernt. Läuft eine Installation oder eine Anmeldung in einem Terminal im Fenster, führt **Zurück** (oder **×** auf ihrem Reiter) zu der Seite zurück, auf der sie gestartet wurde.

Lokale MCP-Server und **Online-MCP-Server** (solche, die über eine Webadresse erreicht werden) sind zwei Abschnitte. **Ordner hinzufügen…** in **Lokale MCP-Server** und in **Skills** fügt einen Ordner hinzu, in dem die App ebenfalls danach sucht — zum Beispiel eine gemeinsam genutzte Platte mit entpackten Apps, die einen MCP-Server anbieten, oder ein Ordner mit eigenen Skills. Die hinzugefügten Ordner stehen neben der Schaltfläche; **×** bei einem beendet die Suche dort (darin wird nichts gelöscht). Was darin gefunden wird, erscheint blass auf der Karte, bis eine Linie es verbindet: Das Setup hat nur Karten für das, was eingerichtet ist.

3.2 Auswählen, was die Karte zeigt

Die Karte zeigt alles, was eingerichtet ist. Die **Zeigen**-Schalter unter der GPU-Karte lassen ganze Bereiche weg — zum Beispiel alle Skills oder die lokalen LLMs und ihre Modelle — und werden gemerkt.

In einem schmalen Fenster (halber Bildschirm oder weniger) wählt die Karte sie selbst, und eine Zeile neben den Schaltern sagt das: Agenten und Anbieter (wer über was arbeitet). Ein dort geklickter Schalter bleibt, bis das Fenster wieder volle Größe hat, und wird nicht gemerkt; das Fenster in voller

Größe behält deine eigene Wahl. Weitere eingeschaltete Bereiche quetschen die Felder nicht zusammen: Die Karte scrollt seitwärts.

3.3 Mit der Karte arbeiten

- **Zeige** mit der Maus auf ein Feld, um es mit allem, womit es verbunden ist, hervorzuheben: ihre Rahmen werden in der Akzentfarbe gezeichnet, der Rest der Karte wird blasser. Sonst hat jedes Feld denselben dünnen grauen Rahmen.
- **Klicke** ein Feld an, um es auszuwählen (✓); `Umschalt`+Klick fügt weitere hinzu. Die ausgewählten Felder und alles, womit sie verbunden sind, behalten den Rahmen in der Akzentfarbe, solange der Zeiger nicht auf einem anderen Feld ist. `Esc` oder **Auswahl aufheben** hebt sie auf.
- Die **Nadel** oben rechts an einem Feld hält seine Verbindungen hervorgehoben, während du woanders hinsiehst.
- **Rechtsklick** auf ein Feld für alles, was seine Karte im Setup bietet: starten oder stoppen, Details, installieren, anmelden, Modelle, aktualisieren, entfernen und mehr.
- ▶ im Feld eines Agenten startet ihn in einem Terminal im Fenster.
- Die Agenten bleiben beim Scrollen immer in der Mitte des sichtbaren Teils der Karte.
- Der aktive Weg jedes Agenten — Agent → genutzter Anbieter → sein Modell — ist fett gezeichnet. Während ein Agent arbeitet, läuft Licht an dieser Linie entlang.
- Die **Nutzungsstatistik-Box**: Während ein Agent arbeitet, stehen kleine Zahlen entlang seiner Linie zum Anbieter, in der Farbe der Linie: wie schnell seine letzte Antwort kam (**t/s**, Token pro Sekunde), seine Kontextgröße (**1M**) und wie viel davon belegt ist (**%**, gelb ab 75 %, rot über 90 %, wie bei den Batterien). Sie stammen aus den eigenen Sitzungsdateien des Agenten (nur Zahlen; Claude Code, Codex und Pi). Was sie zeigt und ihre Textgröße stehen in den Einstellungen.
- Eine rote gepunktete Linie verbindet einen Agenten mit einem Anbieter, für den er eingerichtet ist, den er aber gerade nicht nutzt (kein Modell davon in Gebrauch).
- Nutzen Agenten verschiedene Modelle eines lokalen LLMs, sind die Linien zu diesen Modellen violett: Es hält sie gleichzeitig geladen oder wechselt zwischen ihnen (siehe Modelle gleichzeitig).
- Das Feld eines Online-Agenten hat dieselbe Batterie wie seine Karte im Setup: was von seinen Limits übrig ist; das Feld eines laufenden lokalen LLMs den noch freien GPU-Speicher.
- Ein pro Token bezahlter Agent — mit einem API-Schlüssel angemeldet, also ohne Plan-Limits für eine Batterie — zeigt stattdessen, was seine heute aktiven Sitzungen zu API-Preisen kosten, als kleines **„3,40 $ heute“** auf Kasten und Karte: grün, ab 75 % des Tageslimits gelb und darüber rot (Einstellungen › **Kosten**), jede Minute neu. Seine Details zeigen die ganzen API-Kosten.
- Nutzt ein Programm, das nicht auf der Karte ist, ein lokales LLM, wird die Linie von diesem LLM zum Modell fett in einer eigenen Farbe (Lila oder Cyan, je nachdem, was sich von den anderen Farben des Themes abhebt), und Licht läuft daran entlang. Siehe Von anderen Apps genutzt.
- Hat ein Agent einen anderen gestartet — etwa Claude Code über seine Shell `pi -p …` oder `codex exec …` —, läuft links an der Karte der Agenten ein **Pfeil** vom nutzenden zum genutzten Agenten, und Licht läuft daran entlang; sein Tooltip nennt beide. Ein Agent kann mehrere andere zugleich nutzen, jeden mit einem eigenen Pfeil. Solange er genutzt wird, hat der genutzte Agent auch seine Linien zu seinem Anbieter und weiter zu seinem Modell in der Farbe des Pfeils, und das Licht darauf läuft in dieser Farbe: Du siehst auf einen Blick, welcher Anbieter und welches Modell für wen arbeiten.

- Das Aktualisieren-Symbol (`F5`) in der Kopfzeile sieht erneut nach, was installiert, eingerichtet und angemeldet ist, und führt die Prüfungen eines Starts noch einmal aus (neue Apps, Add-ons, Updates, offene Probleme).
- Bis die erste Linie gezogen ist, zeigt eine Karte über der Karte in drei Schritten, wie es geht; **So geht's ansehen** spielt die Animation aus der Hilfe ab. Nach der ersten Verbindung oder mit × verschwindet sie für immer.
- Zoomen mit **− 100 % +**, `Strg`+Scrollen oder `Strg`+`+` / `-` / `0`.
- Die Felder liegen in einer Karte je Kategorie (Agenten, Online-Anbieter, lokale LLMs, eine Karte je Modellordner, MCP-Server, Skills). Ziehe eine Karte an ihrer Kopfzeile irgendwohin auf die Karte, und ein Feld innerhalb seiner Karte, um die Reihenfolge zu ändern.
- **Klicke auf die Kopfzeile einer Karte**, um sie bis auf ihre Kopfzeile einzuklappen (▸ rechts in der Kopfzeile), und noch einmal, um sie zu öffnen (▾). Die Linien der Felder einer eingeklappten Karte enden an ihrer Kopfzeile, du siehst also weiter, was mit ihr verbunden ist (die Pfeile zwischen Agenten sind ausgeblendet, solange die Agenten-Karte eingeklappt ist). Öffnest du eine Karte, rücken die Karten, die du um sie herum gelegt hast, aus dem Weg.
- Die **Nadel** rechts neben dem Zoom behält das Layout der Karte — wohin du die Karten gelegt hast, die Reihenfolge der Felder und welche Karten eingeklappt sind — nach **Aktualisieren** (`F5`) und einem Neustart. Nicht angeheftet, wie zu Beginn, kommt die Karte bei jedem Aktualisieren und Start im Standardlayout; deine Änderungen bleiben bis dahin. **Standardanordnung** setzt alles auf einmal zurück.
- **Blasse Kästen** in den Karten der MCP-Server und Skills sind auf diesem Computer, werden aber von keinem Agenten genutzt. Zieh eine Linie von einem Agenten zu einem, um ihn zu verbinden (siehe Auf diesem PC gefunden). MCP-Server, die über eine Webadresse erreicht werden, haben eine eigene Karte, **Online-MCP-Server**, neben **Lokale MCP-Server**; beide klappen ein wie jede andere Karte.

> **TIPP** In einem schmalen Fenster (unter etwa 1300 Pixeln, etwa der halbe Bildschirm) scrollt die Karte seitwärts.

3.4 Offene Probleme

Wenn etwas zu beheben ist, erscheint unter den Symbolen der Kopfzeile ein rotes **!**; sein Tooltip sagt, wie viele, zum Beispiel *2 offene Probleme · 4 Vorschläge*. Ein Klick öffnet **Offene Probleme**: je eine Zeile — ein Agent, der nicht angemeldet ist, ein lokales LLM ohne Modelle, eine Verbindung, die nicht funktioniert, wenig freier Platz für Modelle — mit einem Knopf dorthin, wo es behoben wird (die Details des Agenten oder lokalen LLMs, die Karte mit dieser Linie hervorgehoben, Setup, Aufräumen). Dieselben Probleme zeigen sich als rotes **!** auf ihren Karten im Setup. Tipps und was funktioniert stehen auf den Karten und Detailseiten selbst, und wofür das Setup taugt, steht im Setup (**Was du damit machen kannst**). Die Prüfung liest nur; sie ändert, startet oder sendet nie etwas, und sie zeigt nichts, solange die Karte noch leer ist.

3.5 Offene Probleme in der Flutter-Vorschau

In der Flutter-Vorschau öffnet ein **!** im Kopf (rot, wenn etwas zu beheben ist) **Offene Probleme**: alles, was zu beheben ist — zu diesem PC (freier Platz für Modelle), den Agenten, den lokalen LLMs und Modellen oder den Werkzeugen —, jeweils mit einem Knopf, der die Stelle öffnet, an der es behoben wird. **Was du damit machen kannst** zeigt die Anwendungsfälle (Hilfe am PC, Vibe Coding, Bilder, Sprache, Recherche und Schreiben) als **Bereit**, **Fast** oder **Noch nicht**, mit dem, was dafür da ist, und dem nächsten Schritt. **Erneut prüfen** schaut noch einmal nach. Die Prüfungen lesen nur, was die App schon weiß, dazu Grafikkarte, Arbeitsspeicher und freien Plattenplatz; nichts wird geändert. **Updates und Vorschläge** listet neuere Versionen der installierten Agenten und lokalen LLMs, jeweils so geprüft, wie sie installiert wurden: veraltete Werkzeuge von mise, der eigene Release-Kanal eines Agenten (bei Claude Code stable oder latest) und die lokalen LLMs, die die App selbst installiert hat. **Aktualisieren** startet `mise upgrade` oder den eigenen Updater des Agenten in einem eigenen Terminal oder öffnet die Installationsvorschau mit dem neuen Release; das Web wird höchstens alle sechs Stunden gefragt.

3.6 Updates und Vorschläge

Unter den Problemen auf derselben Seite listet **Updates und Vorschläge**, was es zu sagen gibt: neue Programme oder Skills auf diesem PC, was auf diesem PC, aber nicht auf der Karte ist, und Updates für installierte Agenten und lokale LLMs, jeweils mit seinem Knopf (**MCP-Server hinzufügen…**, **Mein Setup**, **Aktualisieren**). Gibt es nur diese und kein Problem, zeigt sich das **!** der Kopfzeile in der Akzentfarbe. Nach Updates wird beim Öffnen des Fensters im Hintergrund gesucht, höchstens alle sechs Stunden. Die Suche selbst installiert nichts; ein Update beginnt erst, wenn du auf **Aktualisieren** klickst.

Ist Einstellungen › **Tägliche KI-Prüfungen** an, stehen hier auch die MCP-Server-Updates, die ein Agent im Web gefunden hat, mit **Release-Seite** und **Mit … erledigen**; sobald die neue Version auf diesem PC ist, bietet die Zeile stattdessen **Verwenden…** an (siehe Gefundene Updates).

3.7 Add-on-Vorschläge in der Flutter-Vorschau

In der Flutter-Vorschau liest die App beim Start im Hintergrund den signierten Add-on-Katalog: aus ihrem Zwischenspeicher, und online nur, wenn **Online nach Add-ons suchen** an ist. Offene Probleme zeigt dann unter **Updates und Vorschläge**: Agenten und lokale LLMs auf diesem PC, deren Add-on ausgeschaltet oder noch nicht installiert ist (**Add-ons holen** oder **Add-ons** öffnet die Add-ons zum Einschalten oder Holen), und installierte Add-ons mit einer neueren Version im Katalog. **Alle aktualisieren** zeigt sie mit ihren Versionen und aktualisiert sie nach der Bestätigung eines nach dem anderen, jedes gegen den Katalog geprüft; eine Meldung sagt, was aktualisiert wurde und was nicht.

KAPITEL 4

Agenten

4.1 Details eines Agenten

Jeder Coding-Agent hat eine Karte im Setup, installierte mit ihrer Version. Klicke die Karte eines installierten Agenten an (MAPRISMA oder ··· › **Details**), oder Rechtsklick auf sein Feld auf der Karte › **Details**. Oben sagt eine Zeile, ob der Agent bereit ist (und wenn nicht, warum) und ob er gerade läuft oder wann er zuletzt benutzt wurde. Darunter ist jeder Bereich eine Karte, die sich zuklappt: Ein Klick auf ihre Überschrift öffnet oder schließt sie. Sie beginnen zugeklappt, jede mit einer einzeiligen Zusammenfassung und Abzeichen; ein Bereich, den du öffnest, bleibt auch auf den nächsten Detailseiten offen.

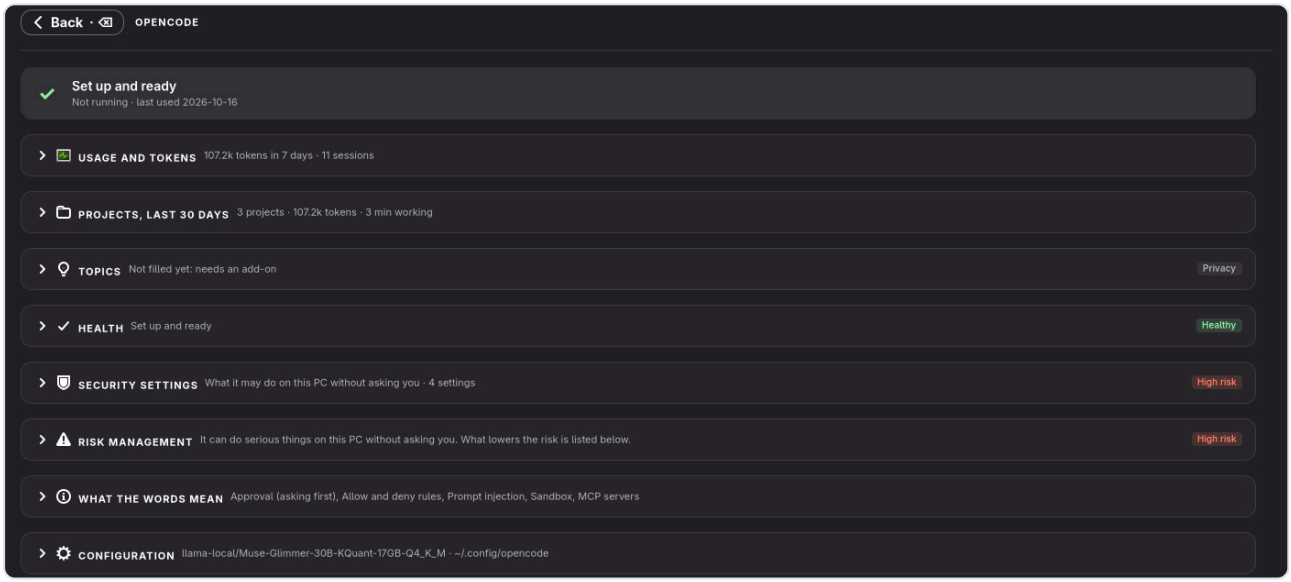


Abbildung 5 · Die Details eines Agenten: jeder Bereich zugeklappt, mit Zusammenfassung und Abzeichen.

Karte	Was sie zeigt
Nutzung und Tokens	Soweit der Agent sie aufzeichnet: Tokens und Sitzungen von heute und den letzten sieben Tagen, das Abo, ein Diagramm der Tokens pro Tag, Limits mit Zurücksetzzeiten und Tokens nach Modell.
API-Kosten / **API-Wert**	Was seine Tokens zu den API-Preisen der Anbieter kosten: heute, die letzten 7 und 30 Tage und pro Modell aufgeteilt in Eingabe, Ausgabe, Cache-Lesen und Cache-Schreiben (Tokens und Betrag). **API-Kosten** für einen pro Token bezahlten Agenten (mit seinem Anteil am Tageslimit), **API-Wert** für einen mit Plan (der Plan deckt es). Siehe API-Kosten.
Projekte, letzte 30 Tage	Pro Projektordner: Arbeitszeit, Sitzungen, Tokens, Subagenten-Läufe, geänderte Zeilen und der API-Wert.

Karte	Was sie zeigt
Themen (Datenschutz)	Worum es in den Sitzungen ging. Das geht über Zahlen hinaus, darum ist sie mit **Datenschutz** markiert und wird nur von einem Add-on gefüllt (siehe unten); das Add-on benennt die Karte, und ihr Abzeichen sagt, wie viel es liest.
Zustand	Jede Einrichtungsprüfung mit einem ✓, oder was als Nächstes zu tun ist: anmelden, sein lokales LLM installieren oder starten, ein Modell herunterladen, MCP-Server oder Skills hinzufügen.
Sicherheitseinstellungen, **Risikomanagement**, **Was die Begriffe bedeuten**	Was der Agent tun darf, ohne dich zu fragen, und was das Risiko senkt (siehe Gesundheits- und Sicherheitscheck).
Konfiguration	Die Einstellungen des Agenten, MCP-Server, Skills und sein Einstellungsordner.

Projekte werden für Claude Code, Codex, Pi, Oh My Pi, OpenCode, Hermes, DeepSeek Harness, Goose, Gemini CLI und OpenClaw gezeigt: Arbeitszeit, Sitzungen, Tokens, Subagenten-Läufe, geänderte Zeilen und was die Tokens zu API-Preisen kosten würden. Die Zahlen stammen aus den eigenen Sitzungsdateien oder der Datenbank jedes Agenten; Text wird nicht gelesen.

API-Kosten. Die Beträge sind eine Schätzung: die Token-Zahlen, die der Agent führt (Eingabe, Ausgabe, Cache-Lesen und Cache-Schreiben, pro Modell), zu den veröffentlichten API-Preisen der Anbieter — Anthropic, OpenAI, DeepSeek, xAI und Google, mit dem Datum, an dem sie übernommen wurden. Eine Sitzung zählt am Tag ihrer letzten Antwort; lokale Modelle kosten nichts. Crush führt keine Token-Zahlen, die sich addieren lassen, darum zeigt seine Karte die Kosten, die Crush selbst meldet. Für einen Agenten, der auf dem PC keine Token-Zahlen führt (Cursor CLI, Antigravity, Muse), sagt die Karte, dass die Kosten nicht bekannt sind; die Kontoseite seines Anbieters hat sie. Beträge erscheinen in US-Dollar oder in Euro zum täglichen Referenzkurs der Europäischen Zentralbank (Einstellungen › **Kosten**).

MAPRISMA Mit dem Add-on **Insights** (siehe Add-ons) zeigt die Karte **Themen** zuerst, welche Themen in den letzten 30 Tagen die meisten Tokens gebraucht haben (**Die meisten Tokens**: Sitzungen mit demselben Titel sind ein Thema, jedes mit seinem Anteil an allen Tokens), dann die Titel, die der Agent seinen Sitzungen selbst gegeben hat, nach Projekt (**Nach Projekt**), und bietet **Mit Claude zusammenfassen…** (wenn Claude Code installiert ist). Ihre Tokens stammen aus den eigenen Sitzungsdateien des Agenten ohne Cache-Lesevorgänge und können daher weit unter denen in **Nutzung und Tokens** liegen, das Cache-Lesevorgänge mitzählen kann. Bevor etwas gesendet wird, siehst du genau, was gesendet wird: die Titel und Zahlen, nie die Gespräche.

4.2 Agenten starten und stoppen

Die letzten 50 Projektordner werden pro Agent gespeichert.

Das **Start**-Untermenü jedes Agenten hat drei durch Linien getrennte Bereiche: **Start** mit dem aktuellen Ordner, **Ordner wählen…** und die durchsuchbare Liste **Zuletzt verwendete Projekte**. Die Liste ist scrollbar und enthält alle über die Karte verwendeten Startordner dieses Agenten, neueste zuerst. Wähle ein früheres Projekt und dann Start. Die Ordnerauswahl startet eine Sitzung im gewählten

Ordner. Ein angehefteter Ordner ist nicht erforderlich: Ohne ihn verwendet Start Einstellungen › **Agenten starten in** oder deinen persönlichen Ordner. Auf der Detailseite zeigt die scrollbare Gruppenkarte **Zuletzt verwendete Projekte** alle gespeicherten Ordner. Ein Klick auf eine Zeile öffnet Start mit diesem Ordner. Die Stecknadel legt den nächsten Standard dieses Agenten fest; pro Agent kann ein Ordner angeheftet werden. Lösen stellt den allgemeinen Standard wieder her. Der Papierkorb entfernt den Eintrag und seine Stecknadel aus dem Verlauf dieses Agenten, niemals den Ordner oder seine Dateien. Untermenü, Verlauf und Stecknadeln gelten für **Hier starten** und **Im Terminal starten**. **Erneut starten** öffnet das Untermenü wieder. Laufende Sitzungen behalten ihren Ordner; abgebrochene Auswahlen und fehlgeschlagene Starts erzeugen keinen Verlaufseintrag.

- ▶ im Feld des Agenten oder **Hier starten** in seinem Menü startet den Agenten in einem Terminal im Fenster, anstelle der Karte. Dafür braucht es VTE (`vte4`).
- **Im Terminal starten** öffnet ihn in einem eigenen Terminalfenster.
- Während er im Fenster läuft: **Terminal zeigen** und **Stoppen**. Stoppen beendet den Agenten und alles, was er gestartet hat, etwa seine MCP-Server.
- **Terminals…** listet, wo der Agent sonst noch läuft: *Hier, in diesem Fenster*, jedes *Terminalfenster* (ein Klick holt es nach vorn) und jede Instanz *Im Hintergrund* — eine ohne Fenster, etwa `claude -p …`, gestartet von einem Skript oder einem anderen Agenten. Ein Klick auf eine Instanz im Hintergrund öffnet einen Terminal-Reiter, der zeigt, was sie schreibt, während sie es schreibt: Fragen, Antworten und die Werkzeuge, die sie nutzt (aus ihrer Sitzungsdatei oder ihrer Ausgabe, wenn diese in eine Datei geht). Der Reiter endet, wenn die Instanz endet. Das ist die einzige Stelle, an der die App Gesprächstext zeigt, und nur für die Instanz, die du anklickst.
- **Beenden…** — auch der kleine Stoppknopf links von ▶ im Feld des Agenten — beendet eine seiner Instanzen außerhalb dieses Fensters. Wähle sie aus der Liste, die sagt, wo und wann jede gestartet wurde. Sie wird gebeten, sich zu beenden (SIGTERM), und beendet, wenn sie nach 3 Sekunden noch da ist. Gestoppt wird nur genau dieser Prozess: Ist er inzwischen beendet und gehört seine Prozessnummer jetzt einem anderen Programm, passiert nichts.

Haben Agenten Terminals im Fenster, zeigt eine Reiterzeile über der Karte einen Reiter pro Terminal mit einem Punkt für *arbeitet*, *läuft* oder *beendet*. Ein Klick auf einen Reiter zeigt dieses Terminal; **×** schließt ein beendetes.

Ein Agent, der irgendwo auf dem Computer offen ist, bekommt einen grünen Punkt und *läuft*, und das Feld des Modells, mit dem er läuft, sagt *in Gebrauch · <Agent>*.

4.3 Netzwerkverkehr

Kasten und Karte eines laufenden Agenten zeigen, was er empfängt und sendet (↓ rein, ↑ raus). Klick auf diese Zeile, oder auf ⇅ in der Kopfzeile für alle laufenden Agenten, um seine Netzwerkseite zu öffnen: **Jetzt**, **Die letzten 5 Minuten** als Diagramm (rein grün, raus blau), **Wohin es geht** — Online-APIs, lokale LLMs, dieser PC und dein lokales Netzwerk, jeweils mit ihrem Anteil — und **Programme**: der Agent selbst und die von ihm gestarteten Programme, etwa seine MCP-Server. Gezählt wird aus den eigenen TCP-Verbindungen des Agenten, solange die App offen ist (ohne Administratorrechte; UDP wird nicht gezählt), und es bleibt auf diesem PC.

4.4 Subagenten

Claude Code und OpenCode können Teile einer Aufgabe an **Subagenten** abgeben. Solange einer der Subagenten eines Agenten arbeitet, zeigt dessen Kasten auf der Karte neben ↑↓ ein kleines Baumzeichen; sind nur fertige gelistet, ist es abgeblendet. Ein Klick darauf öffnet die Seite **Subagenten** des Agenten: eine eigene Karte im Aufbau der großen, links die Karte des Agenten, rechts eine Karte **Subagenten**. Jeder Kasten zeigt die Aufgabe des Subagenten, seinen Typ, den Ordner seiner Sitzung und **arbeitet** oder **fertig vor … Minuten**. Auf den Linien zu den arbeitenden läuft Licht; fertige sind abgeblendet. Mit der Maus über einem Kasten steht die ganze Aufgabe.

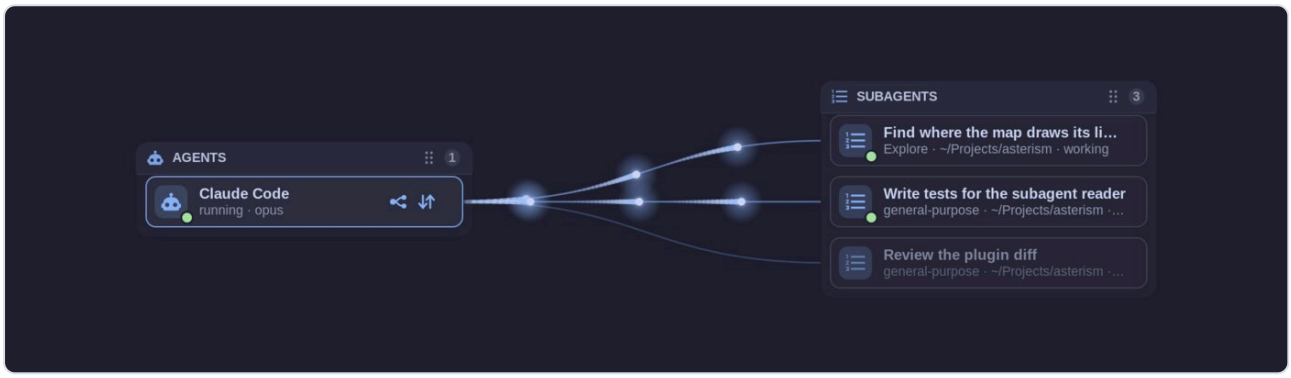


Abbildung 6 · Die Subagenten eines Agenten: zwei bei der Arbeit, einer fertig.

Wie lange fertige Subagenten gelistet bleiben (30 Minuten) und wie viele höchstens (10), steht in Einstellungen › **Subagenten**; arbeitende werden immer gezeigt. Die Seite liest nur Typ und einzeilige Aufgabenbeschreibung jedes Subagenten aus den eigenen Dateien des Agenten (den Sitzungsordnern von Claude Code, der Datenbank von OpenCode), nie, was die Subagenten sagen.

4.5 Agenten, die zusammenarbeiten, in der Flutter-Vorschau

In der Flutter-Vorschau ist ein Agent, der einen anderen gestartet hat und ihn nutzt — etwa Claude Code, das über seine Shell `pi -p …` ausführt —, auf der Karte durch eine Linie mit ihm verbunden, die rechts an den Agenten-Kästen entlang gebogen ist. Licht fließt darauf, solange einer von beiden arbeitet. Erkannt wird das nur über den Prozessbaum (die eigenen Prozesse der Agenten und ihre Eltern); die Linie wird gezeigt, nie bearbeitet. Die Subagenten von Claude Code, Codex, GitHub Copilot, OpenCode und Goose werden wie in der App gelesen.

4.6 Kosten in der Flutter-Vorschau

Die Details eines Agenten zeigen in der Flutter-Vorschau, was seine Tokens zu den API-Preisen der Anbieter kosten: heute, in den letzten 7 und 30 Tagen und je Modell aufgeteilt in Eingabe, Ausgabe, Cache-Lesen und Cache-Schreiben. **API-Kosten** für einen Agenten, der pro Token zahlt (mit dem Anteil am Tageslimit), **API-Wert** für einen mit Abo (das ihn abdeckt). Es ist eine Schätzung aus den Zahlen in seinen eigenen Sitzungsdateien; kein Text daraus wird gelesen. Einstellungen › **Kosten** legt die **Währung** fest (Euro zum täglichen Referenzkurs der Europäischen Zentralbank, einmal am Tag abgerufen), das **Tageslimit** und die Stufen **Gelb ab** und **Rot ab** (Rot immer über Gelb). Die Sitzungsdateien von Claude Code werden dort weitergelesen, wo der letzte Blick aufgehört hat, sodass auch sehr lange Sitzungen ganz zählen; beim ersten Blick auf einem PC mit sehr vielen davon kann es zwei Blicke dauern, bis alles da ist.

Auf der Karte zeigt ein Agent mit Abo-Limits in seinem Kasten eine kleine Batterie mit dem, was vom knappsten Limit übrig ist, und einer, der pro Token zahlt, dort stattdessen die heutigen Kosten; grün, gelb oder rot nach den Stufen aus Einstellungen › Kosten, die Einzelheiten im Tooltip. Beides wird jede Minute neu angesehen. Ist **Bei Rot benachrichtigen** an, bekommen Kosten im roten Bereich eine Desktop-Benachrichtigung am Tag.

4.7 Details in der Flutter-Vorschau

Die Details eines Agenten zeigen auch **Projekte, letzte 30 Tage**: je Projektordner Arbeitszeit, Sitzungen, Tokens, geänderte Zeilen und API-Wert, die Subagent-Läufe je Typ und die aktivste Stunde, aus den Zahlen seiner Sitzungsdateien. Die Karte eines lokalen LLMs sagt, ob es läuft, wie viel GPU-Speicher es belegt und welche Modelle geladen sind, zeigt seine Modelle, Agenten und Adresse, **Nutzung und Tokens**, die Agenten über es geschickt haben (nach Modellname zugeordnet), und unter **Risikomanagement**, wer es erreichen kann. Auf der Karte zeigt ein laufendes lokales LLM den noch freien GPU-Speicher als Batterie. Ein Klick auf den Kasten eines MCP-Servers oder eines Skill-Sets öffnet seine Details: welche Agenten ihn haben und wie jeder ihn startet, aufruft oder liest (Werte von Schlüsseln, Tokens und Query-Parametern werden nie gezeigt), wer ihn noch nicht hat, und die Werkzeuge des Servers vom letzten Nachfragen oder die Beschreibung jedes Skills; **Systemkarte** öffnet die Systemkarte wie bisher.

4.8 Terminal-Tabs, Nutzungsbox und Startmodell in der Flutter-Vorschau

Solange Agenten im Fenster in Terminals laufen, steht über der Karte ein Tab für jeden, mit einem Punkt für **arbeitet**, **läuft** oder **beendet**; ein Klick zeigt das Terminal, das sichtbare ist markiert (Einstellungen: **Terminal-Tabs über der Karte**). Der Kasten eines arbeitenden Agenten auf der Karte zeigt die **Nutzungsstatistik-Box**: wie schnell seine letzte Antwort kam (Tokens pro Sekunde), seine Kontextgröße und wie viel davon belegt ist (gelb ab 75 %, rot über 90 %), aus seinen eigenen Sitzungsdateien gelesen, nur Zahlen; in den Einstellungen steht, was davon erscheint und wie groß. In **Dein erster Agent** schlägt ein Weg mit lokalem LLM das Startmodell vor, das auf deine Grafikkarte passt (**Ein Modell**), und **Herunterladen** startet seinen Download wie die Modellseite.

4.9 Einrichten…

Rechtsklick auf einen Agenten oder ein lokales LLM › **Einrichten…** öffnet dessen eigene Einrichtung in einem Terminal-Tab des Fensters, für das, was die App nicht selbst einstellt — zum Beispiel `hermes setup` oder Ollamas eigenes Menü; LM Studio öffnet sein Fenster. Ein Programm ohne Einrichtungsbefehl zeigt sein `--help`. Wenn es endet, sieht die Karte erneut nach und zeigt, was sich geändert hat, und einem laufenden lokalen LLM wird ein Neustart angeboten.

4.10 Anmelden

Anmelden… im Menü eines Agenten (im Setup oder auf der Karte) zeigt, was der Agent braucht, und seine eigenen Wege zur Anmeldung:

- **Ein Abo oder ein Konto** — die eigene Anmeldung des Agenten läuft in einem Terminal im Fenster (zum Beispiel `claude auth login` oder `codex login`). Dein Passwort oder die Browser-Anmeldung bleibt zwischen dir und dem Agenten; die App sieht es nie. Wo der Agent danach fragt, sagt das Terminal, was einzugeben ist (Pi und Copilot: `/login`).
- **Ein API-Schlüssel** — für Agenten, die einen lesen: Der Schlüssel kommt in ein Passwortfeld, dann direkt in den eigenen Befehl des Agenten (Codex speichert ihn selbst) oder in die Datei, die der Agent liest (Gemini: `~/.gemini/.env`, nur für dich lesbar). Er landet nie in einer Befehlszeile, einem Log oder den Einstellungen der App, und die App behält ihn nicht. Die API-Abrechnung von Claude Code ist dessen eigene Anmeldung bei der Anthropic Console, darum geht kein Schlüssel durch die App.

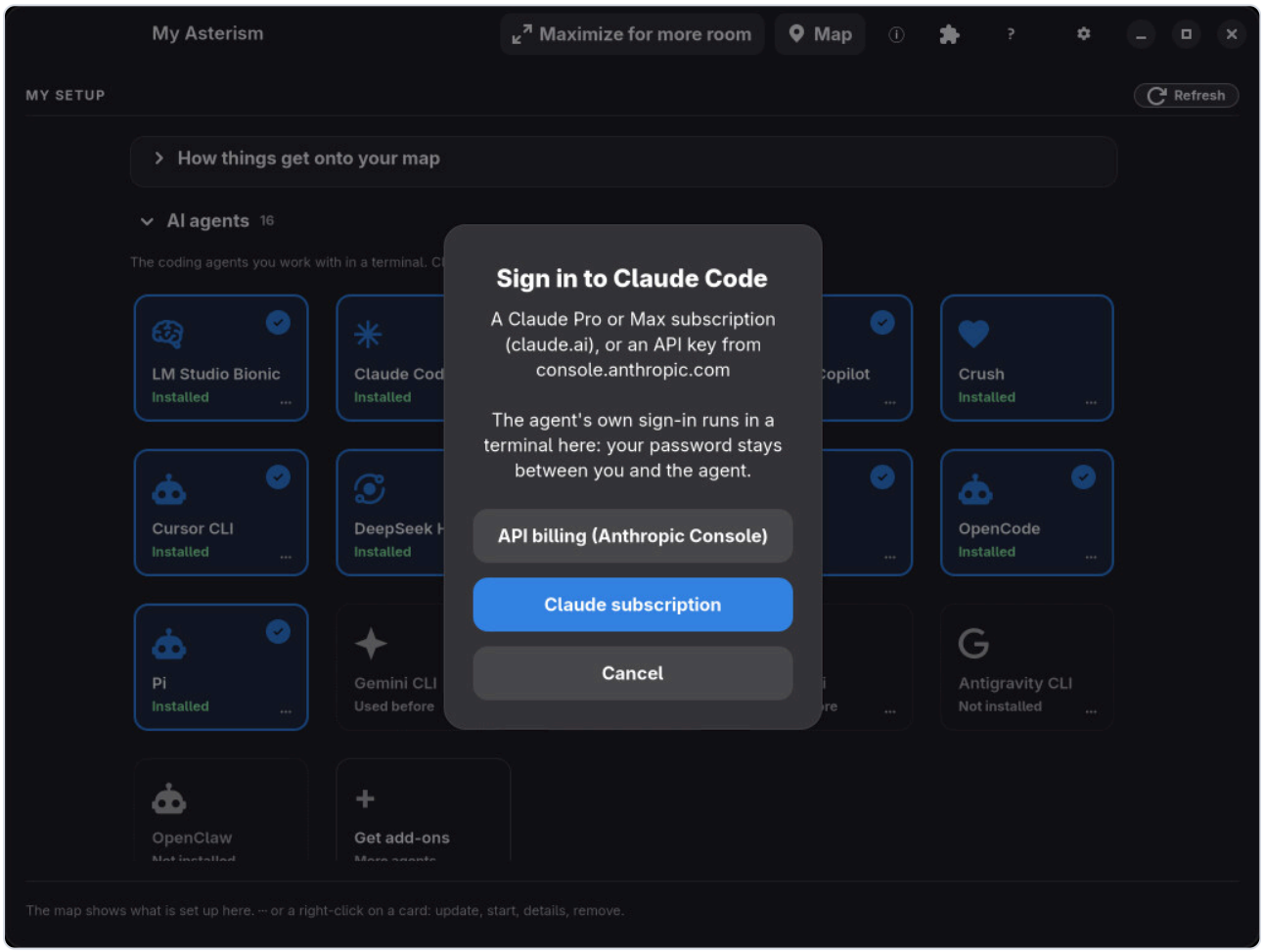


Abbildung 7 · Anmelden bei Claude Code: ein Abo oder API-Abrechnung, beide mit der eigenen Anmeldung von Claude Code.

Danach zeigt die Karte den Online-Anbieter des Agenten und seine Modelle.

4.11 API-Schlüssel in der Flutter-Vorschau

In der Flutter-Vorschau zeigt ein Agent, der einen API-Schlüssel liest, in seinen Details **API-Schlüssel…**. Die Karte sagt, wohin der Schlüssel geht, bevor du ihn eintippst: an den eigenen Befehl des Agenten (Codex: `codex login --with-api-key`) oder in die Datei, die der Agent liest (Gemini: `~/.gemini/.env`, als neue Datei geschrieben, die nur du lesen kannst; andere Zeilen darin bleiben). Das Feld verbirgt, was du tippst, und **Speichern** wird erst bei etwas aktiv, das wie ein Schlüssel aussieht. Der Schlüssel

geht über die Standardeingabe an einen eigenen Worker-Prozess; er landet nie in einer Befehlszeile, einem Protokoll, einer Sicherung oder den Einstellungen der App, und die App behält ihn nicht. Antwortet der eigene Befehl des Agenten, wird seine letzte Zeile gezeigt, ohne den Schlüssel.

4.12 Agenten aktualisieren

Gibt es eine neuere Version eines installierten Agenten, bekommt seine Karte im Setup ein gelbes ↑ **Version**, und **Updates und Vorschläge** (das **!** der Kopfzeile) nennt sie. **Aktualisieren** führt das Update so aus, wie der Agent installiert wurde — zum Beispiel `mise upgrade <tool>` oder `claude update` — in einem eigenen Terminalfenster, damit du es verfolgen kannst.

4.13 Einen laufenden Agenten in der Flutter-Vorschau aktualisieren

Klickst du auf **Aktualisieren** für einen Agenten, der gerade läuft — in einem Terminal-Tab hier oder in einem eigenen Terminal —, sagt die Flutter-Vorschau das zuerst: Ein Update ersetzt das Programm des Agenten, während er arbeitet, deshalb können seine Sitzungen abbrechen oder sich seltsam verhalten, und manche Updater verweigern es. **Abbrechen** lässt alles, wie es ist; lass den Agenten fertig arbeiten und aktualisiere später. **Trotzdem aktualisieren** startet das Update wie gewohnt in einem Terminal.

4.14 Agenten, die die App installieren kann

Vier Agenten lassen sich auf der Karte installieren, aktualisieren und deinstallieren (Rechtsklick auf den Agenten › **Installieren…**):

- **Goose** (`goose` , von der Agentic AI Foundation, früher Block), sein GitHub-Release über mise. Er arbeitet mit vielen Anbietern und mit lokalen LLMs; **Einrichten…** startet `goose configure` .
- **DeepSeek Harness** (`dsh` , Entwickler-Vorschau), von npm. Er kann lokale LLMs nutzen.
- **Antigravity CLI** (`agy` , Google), aus Googles Release-Manifest; der Download wird gegen seine SHA-512-Prüfsumme geprüft.
- **LM Studio Bionic** kann von hier nicht installiert werden (keine veröffentlichte Prüfsumme); hole es von lmstudio.ai. Es nutzt immer die Engine und die Modelle von LM Studio.

4.15 Neue Agenten

Taucht ein Agent auf, den die App noch nicht kennt, fragt sie einmal einen installierten Agenten (Claude Code, sonst Codex), im Web herauszufinden, wo dieser Agent seine Skills und MCP-Server aufbewahrt. Die Antwort wird vor der Verwendung geprüft, und nichts darin wird je ausgeführt. Einstellungen › **Neue Agenten nachschlagen** schaltet das aus.

KAPITEL 5

Lokale LLMs

Eine lokale LLM-Laufzeit lässt Sprachmodelle auf deiner eigenen Grafikkarte laufen, sodass Eingaben den Computer nie verlassen. Die App installiert, startet und stoppt sie für dich.

5.1 Ein lokales LLM installieren

Rechtsklick auf ein graues (nicht installiertes) lokales LLM auf der Karte › **Installieren…**. Du wählst den Programmordner, den Modellordner (vorgeschlagen wird der, den deine anderen lokalen LLMs nutzen) und ob es beim Anmelden startet. Es wird nur für deinen Benutzer installiert, mit einem Benutzerdienst, daher ist kein Passwort nötig. Jeder Download muss zur SHA-256-Prüfsumme passen, die sein Release veröffentlicht, bevor er entpackt oder ausgeführt wird.

Laufzeit	Hinweise
Ollama	Das offizielle Release (mit ROCm für AMD-Karten).
llama.cpp	Der Build deiner Wahl (Vulkan, ROCm, CUDA, CPU), als Modell-Router: jedes GGUF im Modellordner, eines gleichzeitig geladen.
LM Studio	Wird nicht von hier installiert (keine veröffentlichte Prüfsumme); hole es von lmstudio.ai. Einmal installiert, wird es gezeigt und verwaltet.
vLLM	Für NVIDIA-Karten, in einer eigenen Python-Umgebung.
Jan	Sein AppImage, entpackt, mit Jans Engine als Hintergrunddienst für Agenten.
KoboldCpp	Der offizielle Einzeldatei-Build, als Modell-Router.

So installiert, bietet ein lokales LLM außerdem **Aktualisieren…** (die vorige Version bleibt als Rückfall), **Modellordner…** und **Deinstallieren…**, das Programm, Befehl, Dienst und Einstellungen entfernt, aber nie die Modelle. Lokale LLMs, die du selbst installiert hast, werden gezeigt, aber nie verändert.

5.2 Installieren in der Flutter-Vorschau

In der Flutter-Vorschau zeigt eine lokale KI, die noch nicht installiert ist, unter Anbieter und Modelle **Installieren**, und der Leitfaden für den ersten Agenten bietet **mise installieren**, **Node.js installieren** und **Installieren** für die lokale KI an, die sein Weg braucht. Maprisma sucht zuerst das neueste Release und zeigt es auf einer Karte: seine Dateien und ihre Größe, den Programmordner (`~/.local/share/ai-agents-map/<Name>`, ein Ordner je Version), den Modellordner darin und deine Grafikkarte. Bei llama.cpp und KoboldCpp wählst du dort die Variante (Vulkan, ROCm, CUDA oder CPU); eine andere Variante wird erneut gesucht, bevor etwas installiert wird. Eine lokale KI mit Benutzerdienst bietet außerdem **Gleich starten** und **Beim Anmelden starten**.

Installieren lädt die Dateien herunter, prüft jede gegen die Prüfsumme, die ihr Release angibt, und entpackt sie; beim Installieren läuft nichts vom Download (Jans AppImage entpackt sich selbst in einer abgeschotteten Sandbox). Ein Programm, das schon auf anderem Weg installiert ist, oder eine

vorhandene Einstellungsdatei oder ein vorhandener Dienst wird nie ersetzt: die Karte lehnt das ab. Ollama, llama.cpp, KoboldCpp, Jan und mise werden so installiert; LM Studio und vLLM nicht, weil es für sie kein Release mit Prüfsumme gibt.

Jeder Schritt, den du im Leitfaden für den ersten Agenten startest — Installieren, Anmelden, die Testfrage, Anbieter und Modelle, Verbindungen —, bringt dich mit **Zurück** wieder in den Leitfaden, neu geladen, sodass der eben erledigte Schritt als erledigt erscheint.

5.3 Mit den Agenten starten und stoppen

Verbinde einen Agenten einmal mit einem lokalen LLM und starte dann einfach den Agenten — auch bei geschlossener App oder nach einem Neustart. Sein lokales LLM startet mit, und wenn der letzte Agent, der es nutzt, beendet wird, stoppt das LLM etwa fünf Sekunden später, sodass sein Grafikspeicher wieder frei ist.

Das erledigt ein kleiner Hintergrunddienst, `ai-agents-map-providers.service`. Er wird mit der ersten Verbindung zu einem lokalen LLM eingerichtet und entfernt, wenn die letzte wegfällt oder wenn du Einstellungen › **Lokale LLMs mit ihren Agenten starten** ausschaltest.

> **HINWEIS** KoboldCpp braucht ein paar Sekunden zum Starten, länger als manche Agenten warten. Darum startet es beim Anmelden, und wenn der letzte Agent beendet wird, wird nur sein Modell entladen.

5.4 Details eines lokalen LLMs

Klicke die Karte eines lokalen LLMs im Setup an, oder Rechtsklick auf sein Feld auf der Karte › **Details**. Unter den Kacheln (Modelle, Agenten, Adresse) folgen dieselben zuklappbaren Karten wie bei Agenten: **Nutzung und Tokens** — was deine Agenten in den letzten Tagen darüber geschickt haben, pro Agent und Modell, aus den eigenen Sitzungszahlen der Agenten und über den Modellnamen zugeordnet —, dann Zustand, Risikomanagement, Was die Begriffe bedeuten und Konfiguration.

5.5 Leistung

Ein lokales LLM, dessen Einstellungen die App kennt (llama.cpp, Ollama von der App installiert), hat auf seiner Detailseite eine Karte **Leistung**:

- **Variante**, **GPU-Stacks** und **Grafikspeicher**: womit es läuft — ROCm, Vulkan, CUDA oder die CPU (bei llama.cpp aus seiner eigenen Geräteliste gelesen, auch bei einer selbst gebauten Variante), welche dieser Stacks dieser PC hat, und der Speicher der Karte.
- **Kontext pro Modell**: für jedes Modell die Dateigröße, der jetzt eingestellte Kontext, der Kontext, den die Karte dafür fasst — seine Datei plus der Cache des Kontexts, aus dem Header des Modells gelesen — und wofür es trainiert wurde. **!** markiert einen Kontext, der größer ist, als die Karte fasst (das Modell würde nicht laden oder in den RAM ausweichen), ↑ einen, den die Karte größer machen könnte; **Auf … setzen** ändert ihn.
- **… fragen**: Der lokale Helfer (oder der Online-Agent) sieht sich diese Fakten an — nur Namen und Zahlen — und schlägt Änderungen vor: Einstellungen wie Flash Attention, den Cache-Typ des Kontexts, die GPU-Layer, parallele Anfragen und Batch-Größen; Kontexte; eine andere Variante,

wenn sie klar helfen würde. Die App behält nur Vorschläge, hinter denen sie stehen kann (eine bekannte Einstellung mit erlaubtem Wert, ein Kontext, den die Karte fasst), und zeigt jeden mit Begründung. **Übernehmen** schreibt ihn mit Sicherung; die Meldung bietet **Rückgängig** an, oder **Jetzt neu starten**, wenn er erst nach einem Neustart zählt. Eine andere Variante wird nur vorgeschlagen: Installiere sie über das Menü des lokalen LLMs.

Bei Ollama gelten seine Einstellungen für alle Modelle zugleich (die Umgebungsdatei seines Servers).

5.6 Leistung in der Flutter-Vorschau

In der Flutter-Vorschau hat ein lokales LLM mit Einstellungen, die die App abstimmen kann (llama.cpp, Ollama), unter **Anbieter und Modelle** den Knopf **Leistung**. Er zeigt die Variante, als die es läuft, die GPU-Stacks auf diesem PC und den Grafikspeicher, dann **Kontext pro Modell**: die Größe jedes Modells, seinen Kontext jetzt, was die Karte für es fasst und wofür es trainiert wurde. Ein Modell mit mehr Kontext, als die Karte fasst, ist markiert, ebenso eines, für das die Karte noch Platz hat; bei llama.cpp setzt **Auf … setzen** ihn. **Einstellungen für alle Modelle** listet die Einstellungen der Engine mit ihrer Wirkung; eine mit einer Liste von Werten lässt sich dort ändern. Jede Änderung wird zuerst gezeigt und mit einer Sicherung geschrieben, sodass **Rückgängig** sie zurücksetzt; sie zählt nach einem Neustart des lokalen LLMs. Unter **Tipps von einem Agenten** lässt **… fragen** den Agenten für KI-Prüfungen auf diese Fakten schauen (nur Namen und Zahlen, nur über das Web): Er schlägt Einstellungen, Kontexte, einen anderen Build oder Hinweise vor, jeweils mit Begründung. Die App behält nur, was sie vertreten kann — einen erlaubten Wert, einen Kontext, den die Karte fasst, einen Build, den das lokale LLM hat —, und **Übernehmen** geht durch dieselbe Vorschau. Die Tokens sind die des Agenten; der Lauf steht im Smart Check.

5.7 GPU-Speicher und GPU-Speicher freigeben

In der **GPU**-Karte über der Karte zeigt **GPU-Speicher**, wie viel belegt ist, aufgeteilt in lokale LLMs, Desktop und Apps und frei; **GPU-Aktivität** zeigt Last, Temperatur und Leistung. Das Feld jedes lokalen LLMs zeigt, wie viel Speicher es belegt, und geladene Modelle bekommen einen grünen Punkt.

Rechtsklick auf ein lokales LLM für **Starten**, **Stoppen**, Start beim Anmelden und **GPU-Speicher freigeben**: Seine Modelle werden entladen, es läuft weiter und lädt wieder eines, wenn es gefragt wird.

Wenn der letzte Agent, der ein lokales LLM genutzt hat, beendet wird und das LLM seine Modelle noch hält, fragt eine Meldung, ob der Speicher zurückgegeben werden soll: **Freigeben** entlädt sie, **Behalten** lässt sie geladen.

Ein lokales LLM, das GPU-Speicher belegt, bekommt an seinem Feld einen roten Stoppknopf. Er fragt zuerst: **GPU-Speicher freigeben** entlädt seine Modelle; **Stoppen** beendet es. Bei LM Studio heißt der Knopf **Server stoppen**: Nur sein Server endet, die App bleibt mit geladenen Modellen offen. Ein Modell, das ein lokales LLM geladen hat, das aber kein Agent nutzt, ist durch eine durchgehende helle Linie mit ihm verbunden.

LM Studio kann ohne seinen Server geöffnet sein. Sein Feld zeigt dann *Server aus* mit einem gelben Punkt, Agenten erreichen es nicht, und sein Menü bietet **Server starten** (`lms server start`).

5.8 Modelle gleichzeitig

llama.cpp, LM Studio und Ollama können mehr als ein Modell geladen halten. Verbindest du Agenten mit verschiedenen Modellen eines lokalen LLMs, stellt die App ein, wie viele es gleichzeitig geladen hält — so viele, wie seine Agenten nutzen —, mit derselben Vorschau, Sicherung und Rückgängig wie die Verbindung, und bietet einen Neustart an, wenn es läuft, denn es liest die Zahl beim Start. Wo diese Zahl nicht Sache der App ist (eine Systeminstallation oder dein eigener Startbefehl), sagt ein Hinweis, wo sie eingestellt wird.

Passen die Modelle zusammen nicht in den Speicher der GPU, sagt eine Warnung das: Das lokale LLM wechselt sie dann aus, was langsam ist, oder ein Teil eines Modells läuft auf dem Prozessor. Der Start eines Agenten prüft dasselbe: Braucht das Laden seines Modells mehr GPU-Speicher, als frei ist, wirst du vorher gefragt, mit dem, was den Speicher gerade belegt — **GPU-Speicher freigeben, dann starten**, **Trotzdem starten** oder **Abbrechen**.

5.9 Modelle gleichzeitig in der Flutter-Vorschau

In der Flutter-Vorschau schaut die App nach einer übernommenen Änderung des lokalen LLMs oder Modells eines Agenten, wie viele verschiedene Modelle die Agenten jedes lokalen LLMs jetzt nutzen. Wo sie die Zahl setzen darf — die Einstellungsdateien von llama.cpp und Ollama in den App-eigenen Installationen, die Einstellung „eins oder viele“ von LM Studio —, zeigt eine zweite **Vorschau** die Änderung mit ihrer Bedeutung und etwaigen Warnungen; **Übernehmen** schreibt sie mit einer Sicherung, und **Rückgängig** nimmt sie zurück. Eine Zahl, die du in deinem eigenen Startskript festgelegt hast, bleibt, wie sie ist. Sind es mehr Modelle, als geladen bleiben, oder zusammen zu groß für die Grafikkarte, sagen das auch die Offenen Probleme.

5.10 Von anderen Apps genutzt

Lokale LLMs werden nicht nur von Agenten genutzt. Ein Plugin wie Video Trans, eine Chat-App oder dein eigenes Skript kann ebenfalls Anfragen schicken. Die App bemerkt das und zeigt es:



Abbildung 8 · Ein Programm, das nicht auf der Karte ist (hier `curl`), erzeugt mit llama.cpp: die cyanfarbene Linie und das Feld (Theme Tokyo Night). Der eigene Weg von Claude Code zu opus bleibt in der Akzentfarbe.

- **Auf der Karte** wird die Linie vom lokalen LLM zum Modell, mit dem es erzeugt, fett in ihrer eigenen Farbe, Licht läuft daran entlang, und das Feld des Modells bekommt einen Punkt in dieser Farbe und sagt, wer es nutzt, zum Beispiel *in Gebrauch · Video Trans*.
- **Das Rechtsklick-Menü des lokalen LLMs** und seine Seite ⓘ **Details** sagen *In Gebrauch durch Video Trans · jetzt* oder vor wie vielen Sekunden es zuletzt genutzt wurde.
- Die Legende ⓘ zeigt beide Markierungen.

Die App sieht nach, welche deiner eigenen Programme eine Verbindung zum lokalen LLM offen haben. Was sie senden, liest sie nie. Ein Plugin wird mit seinem eigenen Namen genannt, ein Agent mit seinem Namen, alles andere mit seinem Programmnamen. Die App selbst und LLM Monitor, die nur den Status lesen, zählen nicht.

> **HINWEIS** Das Licht zeigt sich nur, während das Programm erzeugt, und verblasst etwa zehn Sekunden nach seiner letzten Anfrage. Ein Programm, das offen, aber untätig ist — Video Trans ohne Sprache zum Übersetzen —, zeigt nichts. Ein Programm bleibt nach seiner letzten Anfrage eine halbe Minute im Menü aufgeführt.

KAPITEL 6

Modelle

Rechtsklick auf ein installiertes lokales LLM › **Modelle…**, oder klicke **Modell hinzufügen** auf **Setup**. Die Seite hat links eine Liste und rechts das gewählte Modell, in drei Reitern.

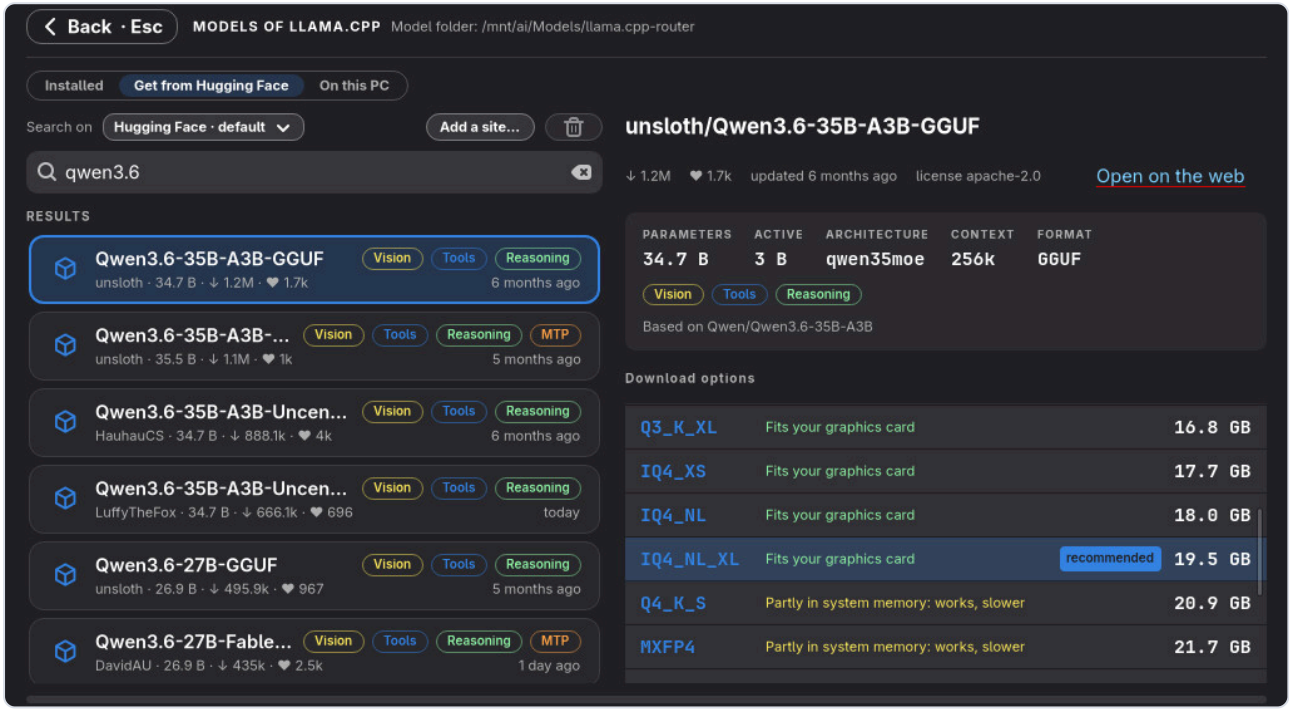


Abbildung 9 · Von Hugging Face holen: was ein Modell kann und welche Größe zu deiner Grafikkarte passt.

6.1 Von Hugging Face holen

Stöbere in den gefragten GGUF-Chatmodellen oder suche. Jedes Ergebnis zeigt Autor, Größe, Downloads, Likes, Alter und was es kann: **Bilder**, **Werkzeuge**, **Denken**, **MTP**. Die Seite des Modells zeigt Parameter, Architektur, Kontext, Lizenz und Modellkarte. **Was der Name bedeutet** erklärt die Teile des Namens, etwa *35B-A3B* oder *Q4_K_M*.

Download-Optionen listet jede Größe mit dem, wie sie passt: in die Grafikkarte, teils in den Arbeitsspeicher (funktioniert, aber langsamer) oder wahrscheinlich zu groß. Die beste Wahl ist schon getroffen; **Herunterladen** holt sie in den Modellordner. Für Ollama, das über den Namen der Größe lädt, werden nur Größen angeboten, die es laden kann: Eine aufgeteilte Datei, eine in einem Unterordner oder eine, die Hugging Face für Ollama ablehnt, ist abgeblendet, mit dem Grund, und wird nie empfohlen.

Downloads laufen im Hintergrund und erscheinen in einem Streifen oben auf der Hauptseite. Bricht die Verbindung ab, wird es erneut versucht und der Download fortgesetzt. Wurde die App während eines Downloads geschlossen, wird er als unterbrochen aufgeführt, mit **Fortsetzen** oder **Verwerfen**.

6.2 Unterbrochene Downloads in der Flutter-Vorschau

In der Flutter-Vorschau wird ein Modell-Download oder ein Ollama-Pull, dessen Verbindung abbricht, nach 5, 15, 30, 60 und 120 Sekunden erneut versucht und macht dort weiter, wo er aufgehört hat. Endet die App vorher, zeigt der nächste Start ein Banner mit jedem unterbrochenen Download, wo er stehen geblieben ist, **Fortsetzen** (es geht mit dem schon geladenen Teil weiter) und **Verwerfen**. Gespeichert wird nur, wie er wieder gestartet wird, in `~/.local/state/maprisma/jobs.json`, nur für dich lesbar; ein Download, der woanders noch läuft, wird nicht doppelt gestartet.

6.3 Einen Agenten fragen, welches Modell passt

Nicht sicher, welches Modell du nehmen sollst? Ist ein Agent für KI-Prüfungen eingerichtet (Einstellungen › **Agent für KI-Prüfungen**), nimmt die Karte **Einen Agenten fragen** unter der Suche eine Frage in deinen eigenen Worten an — zum Beispiel *ein Coding-Modell mit langem Kontext, das zu meiner Grafikkarte passt*. **Fragen** (oder `Strg`+`Enter`) schickt sie zusammen mit deiner Grafikkarte, ihrem Speicher, dem Arbeitsspeicher und den Modellen, die du schon hast, an diesen Agenten, der im Web sucht; ein Drehsymbol und die bisher vergangene Zeit zeigen, dass er arbeitet. Nach ein bis zwei Minuten zeigt die Karte seine kurze Zusammenfassung, und die vorgeschlagenen Modelle erscheinen darunter in der Ergebnisliste mit der Überschrift *Vorgeschlagen von …*: jedes auf Hugging Face gefunden, mit seiner Größe und warum es passt; ein Klick öffnet seine Seite und die Download-Optionen wie bei jedem Suchergebnis. Einen Vorschlag, den Hugging Face nicht hat, nennt die Zeile unter der Überschrift. Er schlägt bis zu 10 Modelle vor; **Vorschläge** rechts in der Zeile mit dem Agenten stellt eine andere Zahl ein (5, 10, 15 oder 20). Es verbraucht die Tokens des Agenten; die Frage wird nicht gespeichert, und die Seite jedes lokalen LLMs fragt für sich.

6.4 Installiert

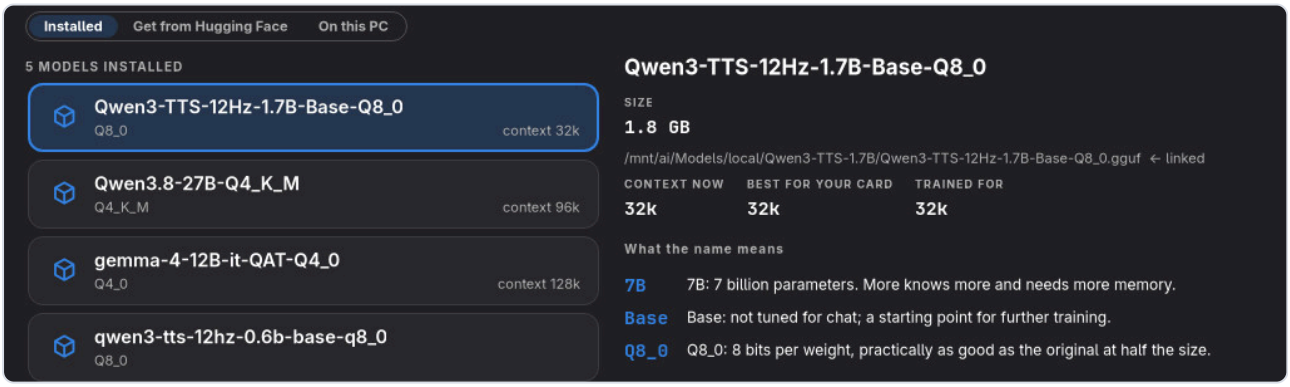


Abbildung 10 · Installiert: Kontext jetzt, am besten für deine Karte und trainiert für; was der Name bedeutet; Entfernen.

Jedes Modell zeigt Größe, Datei und Kontext — *jetzt*, *am besten für deine Karte* und *trainiert für*. **Entfernen** fragt zuerst und verschiebt die Dateien in den Papierkorb, aus dem du sie wiederherstellen kannst. Ein Modell, das nur eine Verknüpfung ist, verliert nur die Verknüpfung.

6.5 Kontextgröße

Der Kontext ist, wie viel Text das Modell auf einmal im Blick behalten kann. Die App berechnet aus der Modelldatei selbst den größten Kontext, der zu deiner Grafikkarte passt. **Kontext setzen** schreibt ihn in die Einstellungen des lokalen LLMs, mit Sicherung und Rückgängig. Agenten brauchen einen recht großen Kontext; verbindest du einen Agenten mit einem LM-Studio-Modell ohne gesetzten Kontext, wird im selben Schritt der beste gesetzt.

6.6 Auf diesem PC

Listet GGUF-Dateien, die das lokale LLM noch nicht liest — in seinem Modellordner, den Ordnern der anderen lokalen LLMs, `~/Downloads` und `~/Models`. **Hinzufügen** macht eine davon zu seinem Modell ohne zweite Kopie (über eine Verknüpfung), außer bei Ollama, das eine eigene Kopie importiert.

6.7 Modelle verschieben und löschen

Rechtsklick auf ein Modell von llama.cpp, KoboldCpp oder LM Studio › **Verschieben nach…**, oder ziehe das Modellfeld auf den Ordner eines anderen lokalen LLMs. Zuerst siehst du, wie viel verschoben wird und welcher Agent das Modell nutzt. Rückgängig verschiebt die Dateien zurück. **Löschen…** fragt zuerst und verschiebt die Dateien in den Papierkorb.

6.8 Ein Modellordner

Einstellungen › **Modellordner** legt einen Ordner für die Modelle aller lokalen LLMs fest. **Modelle aufräumen…** zeigt vorab, wie die Dateien sortiert würden (`<Ordner>/<Herausgeber>/<Modell>/`), und nichts ändert sich, bis du **Aufräumen** drückst. Mit **Alte Pfade funktionieren weiter** (standardmäßig an) bleibt an jedem alten Ort eine Verknüpfung. **Letztes Aufräumen rückgängig machen** stellt alles wieder her.

> **VORSICHT** Beim Verschieben auf eine andere Festplatte werden die Modelle zuerst kopiert und die Originale erst entfernt, wenn die Kopie vollständig ist. Achte darauf, dass auf dem Ziel genug Platz frei ist; sonst lehnt die App das Verschieben ab.

KAPITEL 7

Verbinden durch Zeichnen

Drücke auf ein Feld der Karte und ziehe zu einem anderen, um sie zu verbinden. Jede Änderung zeigt vor dem Schreiben genau, welche Zeilen welcher Datei sich ändern, legt eine Sicherung an und lässt sich über die folgende Leiste rückgängig machen.

7.1 Agent ↔ lokales LLM

Ziehe von einem Agenten zu einem lokalen LLM oder zurück. Wähle das Modell und ob der Agent es gleich nutzt. Um eine Verbindung zu ändern, wähle ihren Agenten aus (oder klicke die Linie an): Jede Linie, die sich verschieben oder entfernen lässt, zeigt an ihrem Ende einen **orangen Griff**. Ziehe den Griff zu einem anderen lokalen LLM — oder ins Leere, um sie zu entfernen; fährst du über einen Griff, steht dort, was das Ziehen bewirkt. Ist ein Agent ausgewählt, macht ein Klick auf ein Modell eines verbundenen lokalen LLMs dieses Modell zum Modell des Agenten.

Unterstützte Agenten: Hermes, Pi, Oh My Pi, OpenCode, OpenClaw und DeepSeek Harness.

7.2 Agent ↔ MCP-Server

Ziehe von einem Agenten zu einem MCP-Server. Die Einstellungen des Servers werden in das eigene Format dieses Agenten übernommen. Unterstützt: Claude Code, Codex, Copilot, Crush, Cursor, Gemini, Goose, Grok, Hermes, Muse, Oh My Pi, OpenClaw, OpenCode und Pi. Einstellungsdateien, die Schlüssel enthalten können, werden immer so geschrieben, dass nur du sie lesen kannst.

7.3 Agent ↔ Skill-Ordner

Ziehe von einem Agenten zu einem Skill-Ordner. Pi und Hermes bekommen den Ordner in ihre Einstellungen; jeder andere Agent bekommt pro Skill eine Verknüpfung in seinem eigenen Skill-Ordner. Ein Ordner, den ein Agent von selbst liest (die meisten lesen `~/.agents/skills`), lässt sich nicht nur diesem Agenten wegnehmen.

7.4 Auf diesem PC gefunden

Bei jedem Öffnen des Fensters (und mit **Aktualisieren**) sucht die App auf diesem Computer nach MCP-Servern und Skill-Ordnern, ohne etwas zu starten: Programme, die einen MCP-Server anbieten, in `~/.local/bin`, `~/Applications`, `/opt` und den im Setup hinzugefügten Ordnern, und Skill-Ordner, die Apps mitgebracht haben oder die in einem hinzugefügten Ordner liegen. Was noch kein Agent nutzt, steht als blasser Kasten ohne Linien auf der Karte. Nichts wird von selbst verbunden.

1. Zieh von einem Agenten zum blassen Kasten.
2. Die Verbindungsseite zeigt, wie der Server gestartet wird — mit dem Befehl, den sein eigener Hilfetext nennt, zum Beispiel mit einem Ordner, den er lesen und beschreiben darf — und genau, was sich in den Einstellungen des Agenten ändert.

3. **Verbinden** schreibt es. Der Kasten ist nicht mehr blass, und die App prüft sofort, ob der Server antwortet (siehe Wenn eine Verbindung fehlschlägt).

Der MCP-Server einer App läuft für sich: Das Fenster der App muss nicht offen sein. Programme in einem AppImage, das noch nicht entpackt ist, stehen nicht auf der Karte; **MCP-Server hinzufügen** › **Auf diesem PC** entpackt sie zuerst.

> **TIPP** Leg Apps mit einem MCP-Server und deine eigenen Skills in je einen Ordner — zum Beispiel auf einer Platte, die mehrere Systeme teilen — und füge diese Ordner im Setup hinzu. Auf einem anderen Computer fügst du dort dieselben Ordner hinzu, und alles darin steht auch auf dessen Karte.

7.5 Lokale Verbindungen in der Flutter-Vorschau

Öffne Verbindungen und dann MCP-Server hinzufügen. Vergib einen Namen, wähle stdio, HTTP oder SSE und die installierten Agenten. Bei stdio stehen Programm und Argumente getrennt: ein Argument pro Zeile; Leerzeichen innerhalb einer Zeile gehören zum Argument. Umgebungsvariablen oder HTTP-Header werden als Name/Wert-Paare eingegeben; Werte sind verdeckt. Die Vorschau zeigt Empfänger und Dateiänderungszahlen, keine Konfigurationswerte. Vor dem Ersetzen gleichnamiger Definitionen erscheint ein Hinweis. Anwenden nutzt Sicherung und Rückgängig. Maprisma startet oder testet den Server hier nicht; Agenten können ihn beim Lesen der gespeicherten Konfiguration starten.

Skills hinzufügen wählt einen vorhandenen Ordner mit Skill-Ordnern, jeweils mit SKILL.md. Wähle den übergeordneten Sammlungsordner und die Agenten. Die Vorschau zeigt Verknüpfungen oder Adaptereinstellungen. Originale bleiben erhalten; Verknüpfungen folgen späteren Änderungen. Skills können ausführbare Skripte enthalten. Bereits automatisch gelesene Ordner brauchen keine neue Verknüpfung. Die Suche prüft bis zu 200 Einträge pro Ordner, eine Kategorieebene tief und insgesamt 1.000 Einträge; wähle bei Bedarf einen kleineren Ordner. Pro Vorschau sind 32 Agenten und 200 Verknüpfungen möglich. Rückgängig entfernt neue Verknüpfungen oder stellt Einstellungen wieder her und erhält spätere Änderungen. Onlinekataloge, Skill-Downloads und MCP-Werkzeugprüfungen gehören nicht zu diesen lokalen Formularen.

7.6 Auf diesem PC gefunden in der Flutter-Vorschau

In der Flutter-Vorschau sucht die App beim Start im Hintergrund, ohne etwas auszuführen, nach Programmen auf diesem PC, die einen MCP-Server anbieten (ihre Dateien enthalten die eigenen Wörter des Protokolls; wie man sie startet, steht in ihrem eigenen Usage-Text), und nach Skill-Sets, die Apps mitgebracht haben. Große Programme werden einmal gelesen und gemerkt. Offene Probleme meldet **Neu auf diesem PC** für das, was noch nicht gezeigt wurde; **MCP-Server hinzufügen…** oder **Skills hinzufügen…**, oder **Auf diesem PC gefunden** unter Verbindungen, listet sie auf, die neuen mit einem ★. **Hinzufügen…** öffnet das Formular, ausgefüllt mit dem Namen und dem Startbefehl oder dem Ordner, zum Prüfen und Ändern vor der Vorschau. AppImages werden von der Suche selbst nie gestartet. **Hineinsehen…** entpackt nur die Programme jedes AppImages, abgeschottet durch bubblewrap (kein Netz, kein Home-Ordner, nur ein temporärer Ordner beschreibbar), liest sie und löscht sie wieder; was gefunden wurde, wird gemerkt. Einen Server hinzuzufügen, der in einem AppImage steckt, fragt erst, ob das ganze AppImage nach `~/Applications/<app>-<version>/` entpackt werden soll (**Entpacken**), auf dieselbe Weise. Ein neueres AppImage einer App, in die schon

hineingesehen oder die schon entpackt wurde, wird nur genannt (**Neue Version gefunden**): Eine Datei mit neuerem Namen beweist kein echtes Update, also wird auch in sie wie in jede andere hineingesehen.

Wird eine App in einer neueren Version gefunden, während Agenten, die Liste der MCP-Server oder ein Link in `~/.local/bin` noch die ältere starten, zeigt die Liste sie mit ihren Versionen und wer sie nutzt. **Aktualisieren…** (nach dem Entpacken des neueren AppImages, falls es eines ist) zeigt eine Änderung für alles in der Vorschau: Jeder Eintrag bekommt das neue Programm (Argumente in den alten Ordner ziehen mit um), die Links zeigen in den neuen Ordner, und der alte Ordner kommt in die Sicherung. Dort bleibt er 7 Tage; Rückgängig stellt alles wieder her.

7.7 Wenn eine Verbindung fehlschlägt

Gleich nachdem eine Verbindung geschrieben ist, prüft die App, ob sie funktionieren wird: ob ein lokales LLM installiert ist und das Modell anbietet, ob ein MCP-Server antwortet, ob die Skill-Verknüpfungen intakt sind. Eine fehlgeschlagene Verbindung bleibt als **rote gepunktete Linie** mit einem **!** auf der Karte; eine mit nur Warnungen ist **gelb gepunktet**. Klicke die Linie oder ihr **!** an für Hinweise:

- **Erneut testen** prüft noch einmal; besteht sie, wird die Linie normal.
- **Erneut versuchen…** öffnet die Verbindungsseite für eine Verbindung, die nie geschrieben wurde.
- **Verbindung entfernen** entfernt sie.

KAPITEL 8

MCP-Server

MCP-Server geben Agenten zusätzliche Werkzeuge. **+ MCP-Server** auf der Karte oder unter den Karten öffnet die Seite zum Hinzufügen: links eine Liste, rechts die Seite des gewählten Servers.

8.1 Woher Server kommen

Reiter	Inhalt
Empfohlen	Die Referenzserver des MCP-Projekts (Filesystem, Fetch, Git, Memory, Sequential Thinking, Time, Everything) und offizielle Server bekannter Hersteller (GitHub, Playwright, Chrome DevTools, Brave Search, Context7), nach Zweck gruppiert und auf ihre neueste Version festgelegt.
MCP-Registry	Eine Suche in der offiziellen MCP-Registry. Jedes Ergebnis nennt seinen Herausgeber so, wie die Registry ihn geprüft hat. Die Suche kann zehn Sekunden oder länger dauern. Die Ergebnisse kommen zu je 20; die Zeile unter der Liste blättert durch sie und zeigt, wo du bist, zum Beispiel **‹ Vorherige 20** · 21 – 40 / 200 · **Nächste 20 ›** (mit */ 40+*, wenn die Registry nicht sagt, wie viele es sind).
Auf diesem PC	Programme auf diesem Computer, die einen MCP-Server anbieten, gefunden durch Lesen ihrer Dateien — gestartet wird nichts. In AppImages wird nur hineingesehen, wenn du auf **Hineinsehen…** klickst; sie werden in bubblewrap eingeschlossen entpackt (kein Netzwerk, kein Home-Ordner), daher muss `bwrap` installiert sein.
Von Hand	Eine Befehlszeile oder eine Webadresse mit Headern.

Nicht sicher, welchen Server du brauchst? Im Tab **MCP-Registry** nimmt die Karte **Einen Agenten fragen** unter der Suche eine Anfrage in deinen eigenen Worten an — zum Beispiel *meine Notion-Seiten lesen und schreiben*. **Fragen** (oder `Strg`+`Enter`) lässt den Agenten für KI-Prüfungen daraus einige Suchwörter machen; die App durchsucht damit die heruntergeladene Registry-Liste wie die normale Suche, und der Agent wählt aus dem Gefundenen die passenden Server, jeweils mit Begründung. Er sucht nicht im Web, sieht nur deine Anfrage und die öffentlichen Registry-Texte der Server und kann nur Server wählen, die wirklich in der Liste stehen. Ein Drehsymbol und die bisherige Zeit zeigen, dass er arbeitet; nach einigen Sekunden bis einer Minute erscheinen die Vorschläge in der Ergebnisliste, bis zu 10 (**Vorschläge** rechts in der Zeile mit dem Agenten: 5, 10, 15 oder 20). Eine Suche, die du danach eingibst, ersetzt sie.

Auf diesem PC sucht auch in den Ordnern, die mit **Ordner hinzufügen…** unter Setup › **Lokale MCP-Server** hinzugefügt wurden: in jedem Ordner selbst und in den App-Ordnern darin, etwa `photocraft-0.16.0/usr/bin`.

8.2 Einen Server einrichten

Die Seite des Servers sagt, wer ihn gemacht hat und wie gut das geprüft ist, was er tut (mit Dingen, die du einen Agenten danach fragen kannst), wie er läuft — als Programm über `npx`, `uvx` oder Docker, oder als Webdienst — und was er braucht, etwa einen Ordner oder einen API-Schlüssel mit einem Link, wo es ihn gibt. Darunter **Einrichten**:

1. Fülle die Felder aus, die der Server braucht.
2. Hake die Agenten an, die ihn bekommen sollen.
3. Klicke **Testen**, um den Server nach seinen Werkzeugen zu fragen.
4. Klicke **Hinzufügen**: Alle angehakten Agenten werden auf einmal geschrieben, mit einer Vorschau und einem Rückgängig.

> **HINWEIS** Geheime Werte wie API-Schlüssel werden in der Umgebung oder in Headern übergeben, nie in einer Befehlszeile, wo andere Benutzer des Computers sie lesen könnten. Ein Server, der ein Geheimnis in seiner Befehlszeile bräuchte, muss von Hand eingerichtet werden.

8.3 MCP-Server in der Flutter-Vorschau

In der Flutter-Vorschau öffnet **MCP-Server finden…** unter **Verbindungen** die Reiter **Empfohlen** (beim Öffnen auf ihre neueste Version festgelegt) und **MCP-Registry** (20 Treffer auf einmal mit **‹ Vorherige 20** und **Nächste 20 ›**; die Liste der Registry wird im Hintergrund geladen, damit die Suche schneller geht). Die Seite eines Servers sagt, wer ihn gemacht hat, was er tut, was er braucht und **Wie er läuft**; der Weg, der auf diesem PC laufen kann, ist **empfohlen**, und ein fehlendes `npx`, `uvx` oder Docker wird genannt. Fülle die Felder aus, die er braucht (geheime sind verdeckt), prüfe den **Name**n, hake die Agenten an und klicke **Vorschau**: Vorschau, **Übernehmen** und **Rückgängig** sind die einer Verbindungsänderung. Nur die eingetippten Werte gehen an den Dienst; der Befehl selbst kommt aus dem Eintrag des Servers und wird dort noch einmal geprüft. Ein Geheimnis, das in einer Befehlszeile landen würde, wird abgelehnt; so einen Server richtest du von Hand ein.

Testen startet den gewählten Weg einmal (oder fragt seine Adresse) und listet die Werkzeuge, die er anbietet, bevor etwas geschrieben wird. Ein schon eingerichteter Server zeigt in seiner Karte unter **Verbindungen Werkzeuge**: die letzte Antwort mit Datum und **Server fragen** (oder **Erneut fragen**), das ihn einmal so startet wie sein erster Agent und danach wieder beendet. Seine **Prompts** stehen auch dabei. Die Antwort bleibt für das nächste Mal; Schlüssel aus seinen Einstellungen gehen privat an den fragenden Prozess und werden nie angezeigt.

8.4 Server verwalten

ⓘ **Details** auf der Karte eines MCP-Servers zeigt die Einstellungsdatei jedes Agenten, den Befehl oder die Adresse und auf Wunsch die Werkzeuge des Servers; Werte von Schlüsseln werden nie gezeigt. **Entfernen** nimmt den Server allen Agenten weg. Server bleiben in der Liste, wenn kein Agent sie mehr nutzt, damit eine Linie sie wieder verbinden kann.

Taucht in `~/Applications` eine neuere Version einer App auf, die einen MCP-Server bereitstellt, nennt sie ein Hinweis. **Aktualisieren…** entpackt die neue Version, richtet jeden Agenten darauf aus und behält die alte sieben Tage als Sicherung. **Nach Updates suchen** (unter **Auf diesem PC**) tut das nach

deinem Klick für alle solchen Apps auf einmal: Es sieht in die neuen Versionen hinein, bietet alle Updates in einer Vorschau mit einer Sicherung und einem Rückgängig an und testet dann jeden aktualisierten Server; ein Hinweis nennt jeden, der nicht antwortet.

KAPITEL 9

Skills

Ein Skill ist ein Ordner mit Anweisungen, die einem Agenten eine bestimmte Aufgabe beibringen. **+ Skills** öffnet eine Seite, die wie die MCP-Seite aufgebaut ist.

9.1 Skills finden

- **Auf diesem PC**: Skill-Ordner, die Apps mitgebracht haben und die noch kein Agent liest.
- **Aus dem Web**: eine Suche im Verzeichnis skills.sh, auf einer anderen Seite mit derselben API oder in einem GitHub-Repository mit Skills. Ein Skill wird aus seinem GitHub-Repository in einen Ordner deiner Wahl geholt, standardmäßig `~/.agents/skills`.

> **VORSICHT** Skills können Skripte enthalten, die ein Agent später ausführen kann. Die Seite zeigt das Repository und listet die Skripte, bevor etwas geholt wird — sieh sie dir an.

Auf diesem PC sucht auch in den Ordnern, die mit **Ordner hinzufügen…** unter Setup › **Skills** hinzugefügt wurden.

9.2 Skills aus dem Web in der Flutter-Vorschau

In der Flutter-Vorschau sucht **Skills im Web finden…** unter **Verbindungen** auf skills.sh. Ein Treffer nennt sein GitHub-Repository und wie oft er installiert wurde; wählst du ihn, werden die Dateien des Skills geholt — höchstens 150 Dateien und 30 MB, keine Links, nichts außerhalb seines Ordners — und keine davon wird ausgeführt. Die Karte zeigt seine Beschreibung, wie viele Dateien er hat und welche Skripte ein Agent später ausführen könnte. Wähle den Ordner, in den er kommt (**Nach**: das gemeinsame `~/.agents/skills` oder einer deiner eigenen Skill-Ordner), hake unter **Auch für** die Agenten an und klicke **Vorschau**: Die neuen Dateien werden aufgelistet, und **Übernehmen** schreibt sie mit einer Sicherung, sodass **Rückgängig** sie wieder entfernt. Ein Skill, dessen Ordner schon da ist, wird nicht überschrieben. Die geholten Dateien warten bis dahin in einem privaten Ordner; jede wird vor dem Schreiben mit dem verglichen, was die Vorschau gezeigt hat.

9.3 Skill-Ordner

Jede Karte eines Skill-Ordners zeigt seine Skills und die Agenten, die ihn lesen. Ein Ordner, der über Verknüpfungen geteilt wird, erscheint einmal, dort, wo er wirklich liegt. ⓘ **Details** listet jeden Skill mit seiner Beschreibung und wie jeder Agent den Ordner bekommt.

KAPITEL 10

Einstellungen und Hilfe

10.1 Flutter-Design und Kartenlichter

In der Flutter-App bietet Einstellungen die Optionen System, Hell und Dunkel mit kleinen Vorschauen. System folgt dem Desktop-Erscheinungsbild und der bestehenden Desktop-Palette; Hell und Dunkel wählen eine feste Helligkeit mit passenden Hintergründen und gut lesbaren Farben. Die Auswahl wird gespeichert und gilt auch für Aufgabenkarten. Die GTK-Designeinstellung bleibt getrennt. Mit System zeigt sich ein neues Desktop-Design innerhalb von zwei Sekunden, auch während die App mit einem Download, einer Installation oder einer Änderung beschäftigt ist.

Die Bildrate der Kartenlichter bietet 20 fps (Standard), 30 und 60. Das ist die maximale Bildrate der fließenden Lichter, nicht das automatische Scanintervall. Die Bewegungsgeschwindigkeit bleibt gleich; höhere Raten ergänzen Zwischenbilder. Die tatsächliche Ausgabe hängt von Display und verfügbarer Renderzeit ab. Lichter aus, reduzierte Bewegung, ein unfokussiertes Fenster oder eine verborgene Karte stoppen weiterhin die Animation. Die Auswahl wird gespeichert.

Die Einstellungen bieten außerdem **Lokale LLMs mit ihren Agenten starten**: Startet ein Agent, der mit einem lokalen LLM verbunden ist, startet sein LLM mit; schließt der letzte solche Agent, stoppt das LLM, und sein Grafikspeicher ist wieder frei (das macht ein Benutzerdienst, auch bei geschlossenem Fenster). **Agent für KI-Prüfungen** wählt, welcher installierte Agent bei der KI-Hilfe und dem Smart Check der System Map zuerst angeboten wird; dafür werden seine Tokens verbraucht. Die Liste zeigt die Agenten, die die System Map dafür gefunden hat.

10.2 Das Popup unter der Leiste auf Omarchy

Auf Omarchy öffnet sich die Flutter-App als Popup: ein schwebendes Fenster oben rechts unter der Leiste, ohne Titelleiste, wie die Popups der Plugins. `Esc` auf der Hauptseite oder ein Klick daneben schließt es; ein neuer Start der App (oder das Leisten-Symbol, das `maprisma --toggle` ausführt) holt es an seinen Platz zurück, mit allem, wie du es verlassen hast. Ein Dateiauswahl-Dialog der App schließt es nicht. Einstellungen › **Fenster** wählt **Automatisch: ein Popup unter der Leiste auf Omarchy** (Standard), **Ein Popup unter der Leiste** oder **Ein normales Fenster**; eine Änderung gilt beim nächsten Öffnen des Fensters. Auf anderen Desktops ist die App ein normales Fenster.

10.3 Nutzungszahlen und Ecken von Omarchy in der Flutter-Vorschau

Öffnen sich in der Flutter-Vorschau die Details eines Agenten, dessen Nutzung Omarchy erfasst (Abo, Limits und wann sie zurückgesetzt werden), fragt die App Omarchy im Hintergrund nach frischen Zahlen, wenn die letzten älter als zehn Minuten oder älter als die letzte Anmeldung des Agenten sind, und zeigt sie dann; höchstens einmal alle zehn Minuten je Agent. Einstellungen › **Ecken** legt die Ecken der Karten, Dialoge und Kästen der Karte fest: **Wie Fenster** folgt Hyprlands Fensterrundung (mit der Zahl der Pixel), **Abgerundet** oder **Eckig**.

10.4 Einstellungen

Das Zahnrad (`Strg`+`,`) öffnet die Einstellungen. `Rücktaste` oder `Esc` führt zurück.

Einstellung	Was sie tut
Sprache	Die Systemsprache oder English, Deutsch, Français, Español, Italiano, Português, Nederlands, Polski.
Aussehen OMARCHY	Die Farben des Omarchy-Themes oder GNOMEs Adwaita-Aussehen mit dem hellen/dunklen Modus und der Akzentfarbe des Systems.
Karten-Animationen	Licht läuft an den Linien der Karte entlang: **Agenten und andere Programme** (die Linie eines arbeitenden Agenten und die Linie eines lokalen LLMs zu einem Modell, das ein anderes Programm nutzt), **Nur meine Agenten** oder **Aus**. Eine kleine animierte Karte unter der Einstellung zeigt, was jede Wahl bewegt.
Nutzungsstatistik-Box	Die Zahlen entlang der Linie eines arbeitenden Agenten (siehe Mit der Karte arbeiten): an oder aus, welche von **Token pro Sekunde**, **Kontextgröße** und **Kontext belegt** sie zeigt, und ihre **Textgröße** (standardmäßig 7 pt).
Subagenten	Für die Seite Subagenten: wie lange fertige Subagenten bleiben (**30 Minuten** als Standard; 10, 30, 60 oder 120) und wie viele höchstens (**10** als Standard; 5, 10, 20 oder 40). Arbeitende werden immer gezeigt.
Kosten	**Währung**: US-Dollar oder Euro zum täglichen Referenzkurs der Europäischen Zentralbank (einmal am Tag von ecb.europa.eu geholt). **Tageslimit** für einen pro Token bezahlten Agenten (Standard **100**; 10 bis 500). **Gelb ab** und **Rot ab**: der verbrauchte Anteil, ab dem eine Kostenangabe oder eine Batterie gelb (Standard 75 %) und rot (90 %) wird. **Bei Rot benachrichtigen** (standardmäßig an): eine Desktop-Benachrichtigung, wenn ein Agent rot wird — einmal für jedes Limit, bis es zurückgesetzt wird, oder einmal am Tag für Kosten.
Terminal-Tabs über der Karte	Ein Reiter pro Agenten-Terminal im Fenster.
Ecken	Wie die Fenster, abgerundet oder eckig.
Lokale LLMs mit ihren Agenten starten	Der Hintergrund-Wächter aus Mit den Agenten starten und stoppen.
Im App-Starter	Der Eintrag im Starter; aus entfernt ihn.
Agenten starten in	Der Ordner, in dem Agenten starten.
KI-Funktionen: **Neue Agenten nachschlagen**	Einen installierten Agenten fragen, wo ein unbekannter Agent seine Einstellungen aufbewahrt (standardmäßig an).
KI-Funktionen: **Tägliche KI-Prüfungen**	Einmal am Tag sucht ein Agent im Web nach neuen Versionen deiner MCP-Server (standardmäßig aus; es verbraucht die Tokens des Agenten). Siehe Smart Check.

Einstellung	Was sie tut
KI-Funktionen: **Agent für KI-Prüfungen**	Claude Code oder Codex: übernimmt die KI-Prüfungen und das Nachschlagen sowie die Aufgaben **Mit … erledigen** und **Mit … beheben**.
KI-Funktionen: **Lokaler Helfer**	Ein Agent mit einem Modell deines lokalen LLMs für die KI-Hilfe, die kein Web braucht (aus, bis du einen wählst). Siehe Der lokale Helfer.
Modellordner	Ein Ordner für alle Modelle, mit **Modelle aufräumen…**.
Add-ons MAPRISMA	**Online nach Add-ons suchen**, die Add-on-Seite und **Setup**.
Über	Die Version, der Autor und die Lizenz; **Website** öffnet das Projekt auf GitHub und **Problem melden** seine Fehlerliste.

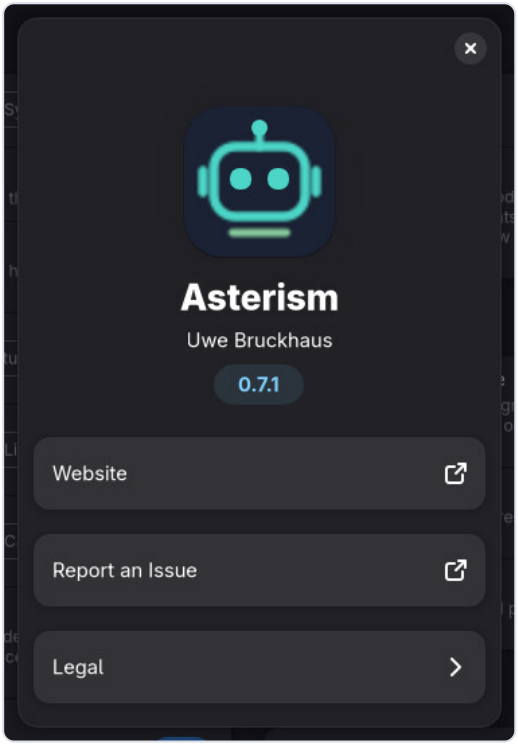


Abbildung 11 · Einstellungen › Über.

10.5 Rote Limits in der Flutter-Vorschau

In der Flutter-Vorschau schaut die App einmal pro Minute auf die Abo-Limits deiner installierten Agenten (eine 5-Stunden-Sitzung, eine Woche …). Ist eines bis zur roten Stufe aus Einstellungen › Kosten verbraucht, sagt es eine Desktop-Benachrichtigung, mit dem verbrauchten Anteil und wann das Limit zurückgesetzt wird: einmal je Limit, bis es zurückgesetzt wird, auch nach einem Neustart. **Bei Rot benachrichtigen** in den Einstellungen schaltet sie ab. Kosten eines Agenten, der pro Token zahlt, zeigt die Vorschau noch nicht, also auch nicht ihre Benachrichtigung.

10.6 Letzte Änderungen und Rückgängig nach einem Neustart

In der Flutter-Vorschau enden die Einstellungen mit **Letzte Änderungen**: die letzten 20 Änderungen, die diese App in die Einstellungen von Agenten geschrieben hat (Verbindungen, MCP-Server, Skills, Sicherheitseinstellungen, App-Updates), die neueste zuerst, mit den betroffenen Dateien. **Rückgängig** stellt wieder her, was seitdem nicht erneut geändert wurde — auch nachdem die App geschlossen wurde oder abgestürzt ist, wenn das **Rückgängig** der letzten Änderung nicht mehr da ist. Eine Änderung, deren Dateien seitdem erneut bearbeitet wurden, hat kein **Rückgängig**. Eine Änderung, die mittendrin stehen blieb (die App wurde beim Schreiben geschlossen), stellt nur die Dateien wieder her, die sie schon geschrieben hatte. Die Sicherungen bleiben, wo sie sind; Rückgängig liest sie nur.

10.7 Hilfe

? (F1) öffnet die Hilfe: **Handbuch öffnen** öffnet dieses Handbuch (die Ausgabe, die mit deiner Version kam, in deiner Sprache) in deinem PDF-Betrachter; **Speicherplatz** (unten); eine Animation zum Einrichten durch Zeichnen; und LLM Monitor, ein begleitendes Leisten-Plugin, das zeigt, was ein lokales LLM tut (eine kleine Animation seines Leisteneintrags und Panels). Beide Animationen sind in den Farben deines Themes gezeichnet und folgen einem Themenwechsel sofort. Die Legende ⓘ in der Kopfzeile erklärt jeden Punkt, jede Linie und jede Markierung der Karte.

Das gefüllte **i** in der Kopfzeile öffnet **Information**: Karten für die anderen Plugins des Autors, gelesen aus dem Omarchy Plugin Marketplace, wenn die Seite öffnet, höchstens einmal pro Stunde (ohne Netzwerk die Liste, die diese Version kennt). Ein Klick auf eine Karte öffnet die Seite des Plugins im Marketplace.

10.8 Hilfe in der Flutter-Vorschau

In der Flutter-Vorschau öffnet **?** im Kopf (oder F1) die **Hilfe**: **Handbuch öffnen** öffnet dieses Handbuch in deiner Sprache (die Kopie, die mit der App kam, sonst die englische) in der PDF-Anzeige des Desktops, und **Wo du anfängst** führt zu **Dein erster Agent**, **Vorhandene KI finden**, **Anbieter und Modelle** und **Verbindungen**.

10.9 Speicherplatz

Speicherplatz: Agenten, lokale LLMs und Modelle zeigt, wie viel Platz dein KI-Setup einnimmt und wo. Er wird bei jedem Öffnen der Hilfe im Hintergrund gemessen (erneut nach zehn Minuten oder mit **Neu messen**) und nur gelesen — von hier wird nichts gelöscht. Die Überschrift zeigt immer die Summen in einer Zeile (*Agenten · Lokale LLMs · Modelle · Insgesamt*); ein Klick klappt den Abschnitt zu. **Festplatten**, **Agenten**, **Lokale LLMs** und **Modellordner** darin beginnen zugeklappt und zeigen jeweils ihre Summe und die größten Einträge; ein Klick öffnet sie.

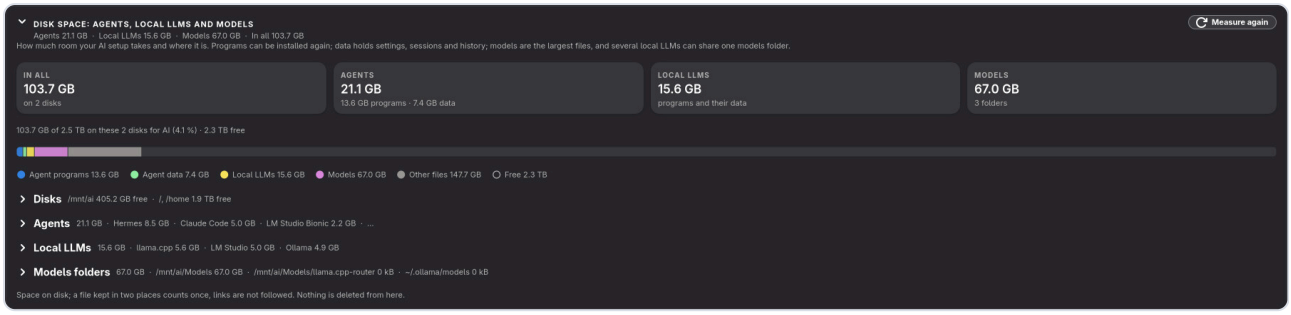


Abbildung 12 · Die Summen, der KI-Anteil am ganzen Plattenplatz und die zugeklappten Abschnitte mit ihren Summen.

- **Kacheln**: alles zusammen, dann Agentenprogramme, Agentendaten (Einstellungen, Sitzungen, Verlauf), lokale LLMs (Programme und ihre eigenen Daten) und Modelle.
- **Balken**: der ganze Platz der Festplatten, auf denen das KI-Setup liegt. Die farbigen Teile sind das KI-Setup, Grau ist alles andere auf diesen Platten, und der leere Rest ist frei; die Zeile darüber gibt den KI-Anteil in Prozent an. Zeige mit der Maus auf den Balken für genaue Größen.
- **Festplatten**: jede Platte mit dem, was das KI-Setup darauf belegt, und wie viel frei ist.
- **Agenten**, **Lokale LLMs**, **Modellordner**: je eine Zeile, die größten zuerst. Öffne eine Zeile, um jeden Ordner mit Größe und Dateianzahl zu sehen und ihn im Dateimanager zu öffnen. Ein Programmordner sagt, wie viele Versionen aufbewahrt werden; für einen Agenten mit mehreren alten Versionen in mise nennt ein Tipp `mise prune`, das die ungenutzten entfernt. Ein Modellordner sagt, welche lokalen LLMs ihn nutzen oder dass er nur Verknüpfungen zu Modellen in einem anderen Ordner enthält.

> **TIPP** Programme lassen sich neu installieren und Modelle neu herunterladen; die Daten der Agenten enthalten deine Sitzungen und Einstellungen. Wird der Platz knapp, sieh dir zuerst alte Programmversionen und Modelle an, die du nicht mehr nutzt.

Größen sind Platz auf der Festplatte: Eine an zwei Orten gehaltene Datei zählt einmal, und Verknüpfungen wird nicht gefolgt.

KAPITEL 11

Smart Check

Mit Smart Check sieht sich ein installierter Online-Agent — Claude Code oder Codex — das KI-Setup dieses PCs an, findet Updates für deine MCP-Server und erledigt Korrekturen für dich, und du siehst, was das an Tokens gekostet hat. Ein lokaler Helfer — ein Agent mit einem Modell deines eigenen lokalen LLMs — kann die Fragen übernehmen, die kein Web brauchen, auf diesem PC und ohne Tokens. Jeder Lauf zeigt seinen Fortschritt in einem Feld über dem Fenster und wird im Protokoll festgehalten. Die zwei Funkelsterne in der Kopfzeile öffnen es; sie zeigen sich nur, solange einer dieser Agenten installiert ist. Einstellungen › **Agent für KI-Prüfungen** wählt, welcher.

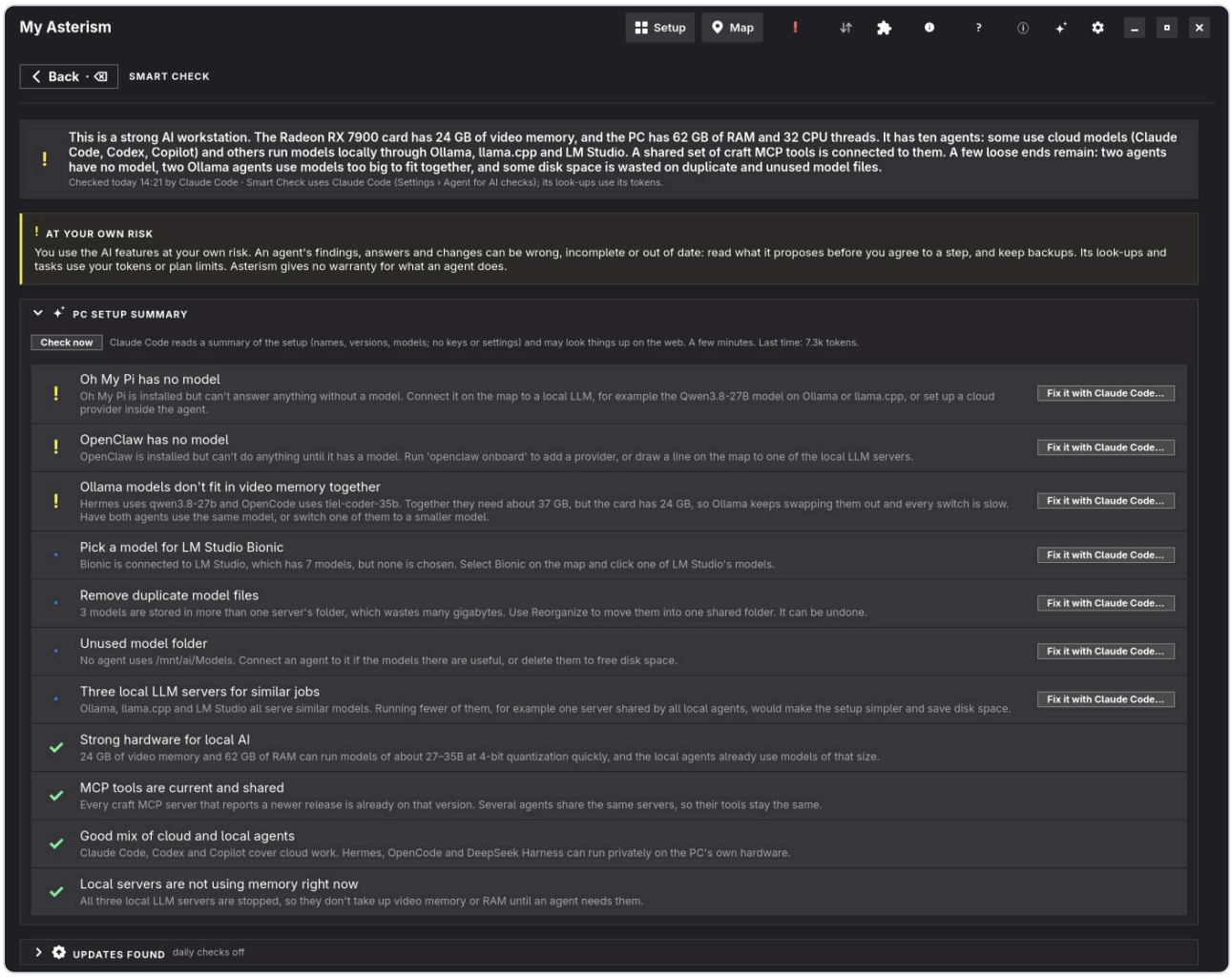


Abbildung 13 · Smart Check nach einer Prüfung durch Claude Code: Probleme (!), Tipps (·) und was gut ist (✓).

> **AUF EIGENE GEFAHR** Die Befunde, Antworten und Änderungen eines Agenten können falsch, unvollständig oder veraltet sein. Lies, was er vorschlägt, bevor du einem Schritt zustimmst, und halte Sicherungen bereit. Sein Nachschlagen und seine Aufgaben verbrauchen deine Tokens oder Plan-Limits. Die App übernimmt keine Gewähr für das, was ein Agent tut. Derselbe Hinweis steht über Einstellungen › **KI-Funktionen**.

11.1 Zusammenfassung des PC-Setups

Jetzt prüfen gibt dem Agenten einen kurzen Text über das Setup — Grafikkarte und Speicher, die Agenten mit ihren Modellen, die lokalen LLMs, die MCP-Server mit ihren Versionen, die Skills und was Offene Probleme gefunden hat — und lässt ihn Dinge im Web nachschlagen. Schlüssel, Header oder Einstellungswerte stehen nicht darin. Das dauert im Hintergrund eine Minute oder länger; die Zeile unter dem Knopf sagt, wie viele Tokens die letzte Prüfung verbraucht hat. Die Antwort ist eine kurze Zusammenfassung und eine Liste von Befunden: **!** Probleme, · Tipps und ✓ was gut ist. Antworten, die nicht standhalten, werden weggelassen.

Mit … beheben neben einem Problem oder Tipp öffnet den Agenten in einem Terminal-Tab im Fenster, mit diesem Befund als Aufgabe: Er sieht nach, ob es so ist, erklärt es und fragt dich vor jedem Schritt, der etwas ändert. Die App startet ihn so, dass er fragen muss — Codex in einer schreibgeschützten Sandbox, die um mehr bitten muss, Claude Code mit jedem Befehl und jeder Bearbeitung auf Nachfragen gestellt, egal was seine eigene Erlaubnisliste sagt — und ohne seine MCP-Server. Der Befund wird als zu prüfende Daten übergeben, nicht als Anweisungen. MCP-Server auf eine neue Version umzustellen bleibt dem **Aktualisieren** der App überlassen (unten), das die Einstellungen aller Agenten auf einmal ändert, mit Sicherung und Rückgängig.

11.2 Gefundene Updates

Neue Versionen deiner MCP-Server, aus zwei Quellen:

- **Die täglichen KI-Prüfungen** (Einstellungen › **Tägliche KI-Prüfungen**, standardmäßig aus): Einmal am Tag sucht der Agent auf den offiziellen Release-Seiten nach neueren Versionen von Servern, deren Version die App erkennen kann — Apps, entpackt in `~/Applications/<app>-<version>/` oder als entpacktes AppImage in einem anderen Ordner (das Programm in `<folder>/<app>-<version>/usr/bin/`), und npm- oder PyPI-Pakete mit fester Version. **Jetzt prüfen** auf dieser Karte fragt sofort, auch solange die täglichen Prüfungen aus sind. Jeder Fund hat **Release-Seite** und **Mit … erledigen**: Der Agent lädt das neue AppImage in einem Terminal-Tab herunter und fragt vor jedem Schritt.
- **Die eigene Suche der App**: Eine neuere Version ist schon auf diesem PC, aber ein Agent startet noch die alte (dasselbe wie das Update auf der Karte des Servers im Setup).

Sobald die neue Version auf diesem PC ist, sagt ihre Zeile das und bietet **Verwenden…** an: Die Update-Vorschau zeigt, welche Agenten auf sie umziehen; **Aktualisieren** ändert sie alle, behält die alte Version mit der Sicherung und testet danach jeden Server. Ein Server, dessen Agenten alle die neue Version starten, fällt aus der Liste.

> **TIPP** Nach einer hier gestarteten Aktion — einem Update oder einer Aufgabe in einem Terminal-Tab — bringt dich **Zurück** wieder zu Smart Check, auf dem neuesten Stand.

11.3 Während ein Agent arbeitet

Jeder Lauf der KI-Hilfe, den du startest — Jetzt prüfen, Einen Agenten fragen auf den Seiten Modelle und MCP-Server hinzufügen, Jetzt prüfen für Updates, Fragen auf der Karte Leistung eines lokalen LLMs — zeigt ein Feld in der Mitte des Fensters: wer gefragt wird (der Agent und sein Modell), seine Schritte (ein Drehsymbol, das zu ✓ wird, oder zu ✕, wenn einer fehlschlug), die bisherige Zeit und am Ende die verbrauchten Tokens.

- **Im Hintergrund** blendet das Feld aus; der Lauf geht weiter, und eine Meldung sagt, wann er fertig ist.
- **Abbrechen** beendet den Agenten sofort (sein Programm und alles, was es gestartet hat).
- Schlägt er fehl, bleibt das Feld mit dem, was schiefging, stehen, bis du es schließt; der Text lässt sich markieren und kopieren.

Nachschlagen im Web (Modellberatung, MCP-Updates) läuft mit dem Web und sonst nichts: ohne die MCP-Server, Connectors, Plugins und Hooks des Agenten.

11.4 Der lokale Helfer

Ein Agent, der mit einem lokalen LLM verbunden ist, kann KI-Hilfe mit einem der Modelle dieses LLMs beantworten: auf diesem PC, ohne Web, ohne Tokens eines Plans und ohne dass etwas den Computer verlässt. Er beantwortet, was kein Web braucht: die Zusammenfassung von Smart Check, welcher MCP-Server passt, und die Tipps auf der Karte Leistung eines lokalen LLMs. Modellberatung und MCP-Server-Updates gehen weiterhin an den Online-Agenten, da sie das Web brauchen.

Die App schlägt einen vor — das größte Chat-Modell, das deine Grafikkarte fasst (Audio-, Embedding- und Tokenizer-Modelle bleiben außen vor) — auf Smart Check, mit **Verwenden**. Einstellungen › **KI-Funktionen** › **Lokaler Helfer** listet jeden Agenten und jedes Modell, das es sein kann, den vorgeschlagenen zuerst, und **Aus**. Nichts wird genutzt, bevor du es wählst. Bevor er antwortet, startet die App das lokale LLM und lädt das Modell (ein Schritt im Fortschrittsfeld). Der Helfer läuft ohne Werkzeuge, ohne MCP-Server, ohne Erweiterungen und ohne Projektdateien. Jeder Agent, dessen Add-on sagt, wie man ihm offline eine Frage stellt, kann der Helfer sein; Pi ist der erste.

11.5 KI-Prüfungen und der lokale Helfer in der Flutter-Vorschau

In der Flutter-Vorschau stehen unter Einstellungen › **KI-Funktionen** die **Tägliche KI-Prüfungen** (standardmäßig aus) und darunter das Ergebnis der letzten Prüfung mit **Jetzt prüfen**, das sofort fragt, auch wenn die täglichen Prüfungen aus sind. Die Prüfung läuft beim Öffnen des Fensters im Hintergrund und wird jede Stunde angesehen; der Agent für KI-Prüfungen wird höchstens einmal am Tag gefragt (nach einer fehlgeschlagenen Suche wieder nach sechs Stunden). Was er findet, steht in Offene Probleme unter **Updates und Vorschläge**: **Release-Seite** öffnet die Veröffentlichung im Browser, **Mit … erledigen** zeigt zuerst die Aufgabe so, wie sie gesendet wird, und startet sie erst nach

Fortsetzen in einem Terminal-Tab, und **Verwenden…** öffnet das auf diesem PC Gefundene, wenn die neuere Version schon da ist. Geprüft werden nur Server, deren Version die App erkennen kann (siehe Gefundene Updates); andere Server werden nicht gesendet.

Der lokale Helfer wird unter Einstellungen › **KI-Funktionen** › **Lokaler Helfer** gewählt; der Vorschlag für deine Grafikkarte ist mit **(empfohlen)** markiert. Ist er gewählt, wird er auf der Karte **Leistung** eines lokalen LLMs unter **Fragen** zuerst angeboten.

11.6 Protokoll

Jeder Lauf der KI-Hilfe wird aufbewahrt, damit du — oder jemand, der sich mit dem Setup gut auskennt — nachsehen kannst, was getan wurde. Jeder Eintrag zeigt, wann er lief, Agent und Modell, das Ergebnis (erfolgreich, fehlgeschlagen, abgebrochen), die Zeit und die Tokens. Geöffnet zeigt er den ganzen Text: die Optionen, mit denen der Agent eingeschlossen wurde, die Schritte, jede Frage genau so, wie sie gesendet wurde, mit der Antwort des Agenten, was die App aufgrund der Antwort geändert hat (mit der Sicherung, für Rückgängig von Hand) und, bei einer Aufgabe in einem Terminal-Tab, was das Terminal am Ende zeigte. **Kopieren** legt einen Eintrag in die Zwischenablage, für einen Bericht oder eine zweite Meinung; **Protokollordner öffnen** zeigt die Dateien.

Das Protokoll behält die letzten 200 Läufe in `~/.local/state/ai-agents-map/ai-help/`, nur für dich lesbar. Fragen enthalten Namen, Zahlen und Einstellungen, nie Schlüssel; der Text eines Terminals enthält, was auf dem Bildschirm stand.

11.7 Token-Statistik

Heute und in den letzten 30 Tagen verbrauchte Tokens, ein Diagramm pro Tag und die Summen nach Agent und nach Plugin (andere Omarchy-Plugins, die denselben KI-Helfer nutzen, erscheinen hier auch). **API-Wert** ist, was die Läufe zu API-Preisen kosten würden, und die Zeile darunter sagt, ob dein Abo das abdeckt oder es über einen API-Schlüssel abgerechnet wird. Eine Aufgabe in einem Terminal-Tab zählt in der eigenen Sitzung ihres Agenten; die Detailseite des Agenten zeigt diese Tokens.

Darunter eine Karte pro Art von Aufgabe — die Setup-Prüfung, MCP-Server-Updates, das Nachschlagen neuer Agenten, Korrekturen — mit ihren Läufen, Tokens und den neuesten Ergebnissen.

KAPITEL 12

Gruppenkarten und Add-ons auf der Karte

12.1 Eine Kategorie erweitern

In der Flutter-Vorschau hat jede Spalte der KI-Karte eine Gruppenkarte. Eine gestrichelte Karte bedeutet: noch nichts eingerichtet; die Fundzahl stammt aus lokaler Erkennung. Platzhalter sind nur Anzeigeelemente, keine verbundenen Knoten oder KI-Eingaben. Erweitern, Doppelklick oder Eingabe auf der fokussierten Karte zeigt die Gruppe allein in der Mitte. Zurück, Esc, Rücktaste oder der Hintergrund stellt Zoom und Auswahl wieder her. Die Systemkarte nutzt dieselben Steuerelemente für Hardware, Datenträger, Programme, Werkzeuge und Ordner.

12.2 Add-ons verwalten

Gruppen zeigen je nach Inhalt Aktiv, Aus, Auf diesem PC gefunden und Nicht installiert. Menü oder Rechtsklick bietet Umschalten, Add-on-Details, eine neuere signierte Version oder Anbieter-Anmeldung über einen installierten Agenten. Details erklären gelesene und gesendete Daten sowie Fähigkeiten. Umschalten gilt sofort mit einer Rückgängig-Meldung. Änderungen sind bei eigenen laufenden Agentensitzungen oder aktiven Installationen gesperrt. Agenten in anderen Terminals blockieren die Add-on-Verwaltung nicht. Erst Add-ons online suchen lädt den signierten Katalog; Erweitern allein tut dies nicht. Installation und Aktualisierung müssen geprüft werden. Neue Add-ons bleiben aus. Entfernen eines Benutzer-Add-ons sichert es und erhält Programme und Daten; Rückgängig gilt bis zum Schließen der App. Mitgelieferte Add-ons bleiben beim Paket.

Nach bestätigtem Entfernen können Sicherungen nach 30 Tagen bereinigt werden. Sicherungen mit aktivem Rückgängig bleiben erhalten; Links, fremde Ordner und Ordner anderer Benutzer werden übersprungen. Rückgängig gilt bis zum Schließen dieser App. Beim ersten Start öffnet Agenten in Willkommen dieselbe Agentengruppe mit festem Einschalten / Später unten. Dies beendet Willkommen nicht und öffnet keine einfache Gruppenansicht.

Aus Datei installieren… nimmt ein Add-on als .zip, das du selbst heruntergeladen hast. Wie bei einem Download aus dem Katalog entscheidet seine eigene Signatur: Die Datei wird zuerst geprüft und wie ein Katalogeintrag gezeigt (Name, Version, Art, was es liest und sendet, und ob es eine installierte Version ersetzt), und installiert wird erst, wenn du bestätigst. Dann wird die Datei noch einmal gelesen und muss noch die geprüfte sein. In der Flutter-Vorschau lassen sich bisher nur Agenten- und Anbieter-Add-ons nutzen.

12.3 Erster Start und vorhandene Abläufe

Willkommen steht über der Karte mit Diesen PC untersuchen / Überspringen und Systemkarte. Hänge vorhandene Datenträger vor der Suche ein. Die Agentengruppe öffnet sich mit markierten Funden. Einschalten / Später bleibt unten sichtbar, während die Liste scrollt. Einschalten ist eine Sammelaktion mit einem Rückgängig; Später erhält die Auswahl. Andere aktive Add-on-Arten bleiben erhalten. Weitere Starts erweitern die Gruppe nicht automatisch. Modelle führt zu Download/Import und

Ordnernutzung, MCP-Server und Skills zu Verbindungen. Neue MCP-Definitionen und neue Skill-Ordner sind noch nicht portiert. Die Rakete öffnet weiterhin den Assistenten für den ersten Agenten. Die Gruppen ersetzen die Seiten Setup und Add-ons.

KAPITEL 13

Die Flutter-KI-Karte bearbeiten

Ziehe einen Agentenkasten zu einem Anbieter, MCP-Server oder Skill-Satz oder zeichne in Gegenrichtung. Klicke einen Agenten und dann ein Modell eines verbundenen Anbieters an. Eine Linie vom Anbieter zu seinem Modell öffnet die Agentenauswahl; lokale Anbieter können, falls unterstützt, auch ohne Agent ein Modell laden. Jede Konfigurationsänderung führt über Vorschau, Anwenden, Fortschritt und Ergebnis mit Rückgängig. Abbrechen schreibt nichts.

Wähle einen Kasten, um orange Verbindungshaken zu zeigen. Ziehe einen Haken zu einem passenden Anbieter oder Modell oder auf leere Fläche zum Trennen. Ein Modellhaken des Anbieters schaltet alle konfigurierten Nutzer gemeinsam um oder trennt sie und entlädt, sofern unterstützt. Beschäftigte Laufzeitsysteme verweigern das Entladen. Online-Verbindungen verwenden den registrierten An- oder Abmeldebefehl im bestätigten Terminal.

Der Mauszeiger hebt Nachbarn hervor. Umschalt-Klick ergänzt die Auswahl, mit höchstens einem Agenten. Anheften hält einen Kasten hervorgehoben. Klappe Karten mit dem Pfeil ein, verschiebe ihre Kopfzeile oder nutze das Kartenmenü. Ziehe den Sortiergriff oder wähle Nach oben / Nach unten. Layout und Pins werden getrennt von Verbindungen gespeichert; Aufgabenkarten erhalten Auswahl und Zoom.

Mit der Tastatur: Aktionen am Kasten öffnen, Verbinden oder Verbindung verschieben wählen und den Zielkasten aktivieren. Trennen entfernt die vorgemerkte Verbindung; Abbrechen oder Esc verwirft sie. Pfeile, Bewegungsmenüs, Anheften und Sortiermenüs ersetzen Ziehgesten. Öffnen zeigt die bisherige Verwaltung. Agentenschaltflächen starten im Standardordner, zeigen oder stoppen eine eigene Konsole oder bieten eine externe Instanz zum bestätigten Stoppen an.

Alle Lichter, Agentenlichter und Lichter aus steuern geschätzte Aktivität. Lichter stoppen ohne aktive Linie, bei verborgener Karte, Fokusverlust oder reduzierter Bewegung. Fehlerlinien zeigen begrenzte Hinweise mit Reparieren oder Trennen. Rückgängig stellt Konfiguration und Fehlermetadaten wieder her; Laden, Entladen und An-/Abmelden werden nicht rückgängig gemacht. Teilfehler behalten die Sicherung und Rückgängig.

KAPITEL 14

Aufgabenkarten auf der Karte

Verbindungen, Agentenaktionen, Modelllöschung und Add-on-Änderungen öffnen in der Flutter-Vorschau eine Aufgabenkarte. Ist der Kartenbereich mindestens 900 logische Pixel breit und sind die betroffenen Elemente bekannt, erscheint links eine schreibgeschützte Karte. Sie zeigt diese Elemente und ihre direkten Nachbarn mit Linien zur Aufgabenkarte. Leere Spalten entfallen. Die Hauptkarte behält Zoom, Position und Auswahl. Die Anbieterübersicht nutzt die volle Breite; einzelne Laufzeitaktionen zeigen eine halbe Karte, wenn ihre Kartenelemente bekannt sind.

Prüfen Sie die vorgeschlagenen Änderungen vor der Bestätigung. Dieselbe Karte zeigt Fortschritt und Ergebnis; nutzen Sie Rückgängig, wenn angeboten. Zurück oder Esc verwirft eine Vorschau oder schließt eine fertige Karte. Laufende Arbeiten lassen sich damit nicht schließen; nur abbrechbare Vorgänge zeigen Abbrechen. Modellsuche und Downloads, Add-on-Katalog und Erstagenten-Assistent nutzen die volle Breite. Das gilt auch für schmale Fenster und Aufgaben ohne bekannte Kartenelemente. Einstellungen, Terminals, Nutzung und Unteragentendetails behalten ihre bisherigen Ansichten.

KAPITEL 15

Systemkarte

15.1 Erster Start und separate Datenträger

Flutter-Entwicklungsvorschau: KI-Karte und Systemkarte behalten Zoom und Auswahl. Hänge vorhandene Datenträger vor Diesen PC untersuchen ein. Die begrenzte Suche liest lokale Metadaten, führt keine gefundenen Programme aus und sendet nichts. Überspringen lässt die Datenträgersuche aus. Hardware, Datenträger, Agenten, Laufzeiten, Werkzeuge und Ordner erscheinen gemeinsam. Fehlende Datenträger bleiben sichtbar.

15.2 Gespeicherte Karte und Aktualisierung

Die private gespeicherte Karte erscheint sofort. Start und Aufruf aktualisieren den Status ohne erneute Vollsuche oder Größenmessung. Aktualisieren durchsucht eingehängte Datenträger und misst Ordnergrößen. Nach der ersten Suche werden neu eingehängte Datenträger einmal durchsucht. Bei sichtbarer Systemkarte erfolgen Einhängeprüfungen höchstens alle 15 Sekunden; Hintergrundintervall, Nur manuell und Fokus-Pause gelten auch hier. Versionsprüfungen müssen ausdrücklich bestätigt werden.

Ist ein Datenträger dauerhaft entfernt, wähle seinen grauen Knoten und Diesen Datenträger vergessen. Die Bestätigung entfernt nur gespeicherte Karteneinträge, keine Dateien oder Einstellungen. Konfigurierte Ordner können beim Aktualisieren wieder erscheinen. Gefundene Laufzeiten zeigen Startpfad (einschließlich current) und aufgelösten Build getrennt. Größen folgen bekannten Wrapper-Zielen; nicht zuordenbare Programmgrößen bleiben verborgen. Bei einem ausdrücklich gewählten Einstellungsprofil liegt system-map.json neben settings.json. Die Standardkarte nutzt den Anwendungscache. Neue dauerhafte Rückgängig-Protokolle liegen immer unter model-moves/ neben settings.json; vorhandene Protokolle im bisherigen Standardcache bleiben lesbar. Bewahre alte Protokolle bis zum Rückgängigmachen ihrer Verschiebungen auf.

15.3 Verbinden, Verschieben und Rückgängig

Wähle gefundene Komponenten zum Prüfen, Neu verbinden oder Ordner verwenden. Ziehe eine Laufzeit auf einen Modellordner für die Verbindungsvorschau oder einen Ordner auf die Laufzeit der KI-Karte. Bestätigte Verbindungen schreiben Konfiguration und Sicherungen, starten aber keine Dienste. Ziehe einen GGUF-Ordner auf einen Datenträger für die Verschiebevorschau. Laufende Leser blockieren dies; zwischen Datenträgern wird zuerst kopiert und geprüft. Unterstützte Verknüpfungen und Verweise werden angepasst. Rückgängig schützt neue Dateien. Eigene Wrapper und nicht unterstützte Speicher brauchen einen geprüften manuellen oder KI-Plan. Einhängevorschläge sind nur Text; fstab wird nie verändert.

15.4 Analyse und KI-Hilfe

Hardwarezustand, GPU-Hinweise und Befunde erscheinen an Knoten. Dieses System mit KI prüfen erfordert Zustimmung zu Zusammenfassung und Prompt: Agent und Online-Modellanbieter erhalten Ordnerpfade; Tokens können verbraucht werden. Mit KI beheben und Einrichtung vereinfachen öffnen geschützte Aufgaben zur Prüfung. Zugangsdaten werden nicht beigefügt. Nutzung auf der KI-Karte zeigt letzte Läufe und Tokenzahlen. Prüfe Duplikate und ungenutzte Dienste vor dem Zusammenlegen und behalte Sicherungen.

KAPITEL 16

Gesundheits- und Sicherheitscheck

Ein Coding-Agent liest und schreibt Dateien und führt Befehle auf deinem Computer aus. Wie viel er tun darf, *ohne dich zu fragen*, steht in seinen eigenen Einstellungen, mit anderen Begriffen bei jedem Agenten. Der **Gesundheits- und Sicherheitscheck** bringt das zusammen: Auf der Detailseite jedes Agenten und jedes lokalen LLMs zeigen vier Karten, ob er funktioniert, was er darf, was das in einfachen Worten bedeutet und was das Risiko senkt.

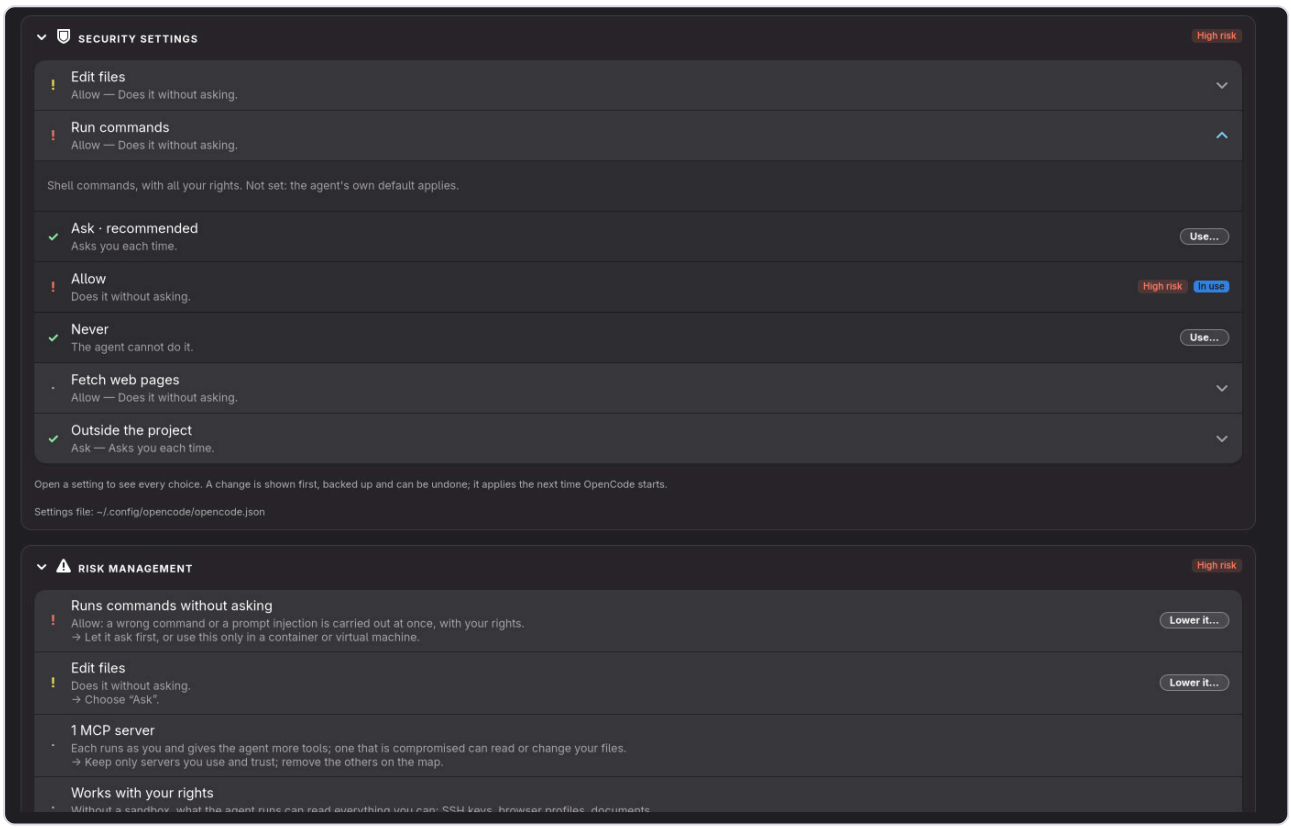


Abbildung 14 · Die Sicherheitseinstellungen von OpenCode mit geöffnetem **Befehle ausführen** und der Anfang seines Risikomanagements.

16.1 Die Karten lesen

Die Abzeichen auf den zugeklappten Karten fassen es zusammen: **Gesund**, **Bitte ansehen** oder **Nicht eingerichtet** bei Zustand, und **Geringes Risiko**, **Mittleres Risiko** oder **Hohes Risiko** bei Sicherheitseinstellungen und Risikomanagement. Öffne eine Karte für die Einzelheiten:

Karte	Was sie zeigt
Zustand	Jede Einrichtungsprüfung mit einem ✓, oder was als Nächstes zu tun ist: installiert, angemeldet, sein Modell und lokales LLM, MCP-Server und Skills.

Karte	Was sie zeigt
Sicherheitseinstellungen	Die Einstellungen des Agenten, die entscheiden, was er selbstständig tun darf, jede mit ihrem jetzigen Wert und einem Satz dazu, was das bedeutet. Das Zeichen davor nennt das Risiko: ✓ sicher, · gering, ein gelbes **!** mittel, ein rotes **!** hoch.
Risikomanagement	Was die Einstellungen zusammen bedeuten, das Ernsteste zuerst, jeweils mit dem Grund und → dem, was es senkt. Vorhandene Schutzmaßnahmen stehen mit einem ✓ da.
Was die Begriffe bedeuten	Sandbox, Freigabe, Erlauben- und Verbieten-Regeln und die anderen Begriffe der Karten, erklärt (siehe Die Begriffe erklärt).

Erlauben- und Verbieten-Regeln und vertrauenswürdige Ordner des Agenten (zum Beispiel die Liste `permissions.allow` von Claude Code) stehen unter den Einstellungen, eine Regel pro Zeile.

16.2 Eine Einstellung ändern

1. Klicke eine Einstellung an, um sie zu öffnen. Du siehst, was sie steuert, und jede Wahl mit ihrem Risiko; die genutzte ist mit **In Gebrauch** markiert, die sicherere Vorgabe mit **empfohlen**.
2. Klicke **Nutzen …** neben einer Wahl oder **Senken …** neben einem Risiko. Eine Seite zeigt die Änderung in Worten (alt → neu), das Risiko vorher und nachher und **Was sich ändert**: die Zeilen der Einstellungsdatei des Agenten.
3. Klicke **Ändern**. Die Datei wird gesichert und geschrieben; eine Meldung bietet 30 Sekunden lang **Rückgängig**. Der Agent nutzt die neue Einstellung beim nächsten Start.

> **HINWEIS** Nur der eine Schlüssel ändert sich; Kommentare und alles andere in der Datei bleiben, wie sie sind. Eine Einstellung, die mehr als eine Zeile in einer Datei braucht — etwa Hermes, das seine Befehle in Docker ausführt —, wird gezeigt und erklärt, mit dem Ort, wo sie zu ändern ist, aber nicht von hier geändert.

16.3 Sicherheit in der Flutter-Vorschau

In der Flutter-Vorschau haben die Details eines installierten Agenten **Sicherheitseinstellungen**. Die Karte zeigt das Risiko-Abzeichen, jede Einstellung mit ihren Möglichkeiten in einer Liste (ein farbiger Punkt zeigt das Risiko jeder Wahl, die sicherere Vorgabe ist **empfohlen**; eine Einstellung, die die Datei nicht nennt, sagt, dass die eigene Vorgabe des Agenten gilt), die Erlauben- und Verbieten-Regeln, die Einstellungsdatei und das **Risikomanagement** mit **Senken …**, wo eine Einstellung ein Risiko senkt. Ein anderer Wert oder **Senken …** öffnet die **Vorschau**, die auch eine Verbindungsänderung nutzt: die Zeilen der Einstellungsdatei, die sich ändern. **Übernehmen** sichert die Datei und schreibt sie, danach mit **Rückgängig**; nur der eine Schlüssel ändert sich, Kommentare und der Rest der Datei bleiben, wie sie sind.

16.4 Zustand in der Flutter-Vorschau

In der Flutter-Vorschau haben die Details eines installierten Agenten und jedes lokale LLM unter **Anbieter und Modelle** eine Karte **Zustand**: **Gesund**, **Bitte ansehen** oder **Nicht eingerichtet**, ein Satz und jede Prüfung — installiert, angemeldet (nur ob eine Zugangsdatei oder ein Schlüssel da ist, wird geprüft), Modell, seine lokalen LLMs, fehlgeschlagene Verbindungen, MCP-Server und Skills; bei einem lokalen LLM, ob es läuft, seine Modelle und die verbundenen Agenten. Eine Prüfung, die nicht in Ordnung ist, sagt, was als Nächstes zu tun ist, wo es einen gibt mit einem Befehl zum **Kopieren**. Die Karte öffnet sich von selbst, wenn etwas zu tun ist.

16.5 Was bei jedem Agenten geprüft wird

Agent	Gezeigte und änderbare Einstellungen	Datei
Claude Code	Berechtigungsmodus (Zuerst fragen, Nur planen, Nur Erlaubtes, Ohne Fragen bearbeiten, Automatische Prüfung, Alle Prüfungen umgehen), Sandbox für Befehle, Umgehungsmodus sperren; Erlauben-/Fragen-/Verbieten-Regeln	`~/.claude/settings.json`
Codex	Wann Codex fragt, Sandbox (nur lesen, Projektordner, keine), Netzwerk in der Sandbox; vertrauenswürdige Ordner	`~/.codex/config.toml`
GitHub Copilot	Berechtigungsmodus (Zuerst fragen, Automatische Prüfung, Alles erlauben); vertrauenswürdige Ordner, Seiten ohne Nachfrage	`~/.copilot/settings.json`
Cursor CLI	Freigabe, Sandbox, Netzwerk in der Sandbox, Websuche ohne Nachfrage; Erlauben-/Verbieten-Regeln	`~/.config/cursor/cli-config.json`
Hermes	Gefährliche Befehle (immer fragen, automatische Prüfung, nie fragen), Geheimnisse verbergen; wo Befehle laufen (nur gezeigt)	`~/.hermes/config.yaml`
OpenCode	Fragen, erlauben oder nie für Dateien bearbeiten, Befehle ausführen, Webseiten abrufen und Arbeiten außerhalb des Projekts	`~/.config/opencode/opencode.json`
Pi	Seine Werkzeuge: lesen, schreiben und Befehle ausführen, oder nur lesen. Pi fragt grundsätzlich nie, darum sind seine Werkzeuge seine Berechtigungen.	`~/.pi/agent/settings.json`

Für andere Agenten zeigt die Karte ihren Zustand und ihre MCP-Server und sagt, wo in den eigenen Einstellungen des Agenten nachzusehen ist. Für ein **lokales LLM** zeigt sie, wer es erreichen kann: nur dieser PC (`127.0.0.1` , wie es sein soll) oder jedes Gerät in deinem Netzwerk.

16.6 Das Risiko steuern

Die Risikostufe ist der ernsteste Punkt, den die Karte findet:

- **Hohes Risiko** — er kann ernste Dinge tun, ohne zu fragen: jeden Befehl ausführen oder ohne Sandbox und ohne Rückfragen arbeiten. In Ordnung in einem Wegwerf-Container oder einer virtuellen Maschine; auf deinem eigenen PC lass ihn zuerst fragen.
- **Mittleres Risiko** — manches geschieht ohne Frage, etwa das Bearbeiten von Dateien im Projekt, oder eine automatische Prüfung entscheidet für dich.
- **Geringes Risiko** — er fragt vor allem, was zählt, oder kann es gar nicht.

Gute Gewohnheiten, egal welche Einstellungen:

- **Bei Befehlen zuerst fragen** auf deinem eigenen Computer; einzelne, harmlose Befehle (etwa `git status`) mit einer Erlauben-Regel freigeben statt die ganze Shell.
- **Die Sandbox einschalten**, wo der Agent eine hat (Claude Code, Codex, Cursor CLI), und das Netzwerk darin aus lassen, solange keine Aufgabe es braucht.
- **Agenten im Projektordner starten**, nicht in deinem persönlichen Ordner, und Schlüssel und Passwörter davon fernhalten.
- **Wenige MCP-Server**: nur die, die du nutzt und denen du vertraust. Jeder läuft mit deinen Rechten.
- **Unbeaufsichtigte Arbeit** (Umgehungsmodus, „YOLO“, Pi mit allen Werkzeugen) in einem Container oder einer virtuellen Maschine laufen lassen.
- **Lokale LLMs auf `127.0.0.1`**: Ihre Server haben kein Passwort.

> **VORSICHT** Eine automatische Prüfung (der Modus *auto* von Claude Code, *assisted* von Copilot, *auto-review* von Cursor, *smart* von Hermes) ist eine zweite KI, die jeden Schritt beurteilt. Sie spart Rückfragen, aber sie ist ein Urteil, keine Mauer.

16.7 Die Begriffe erklärt

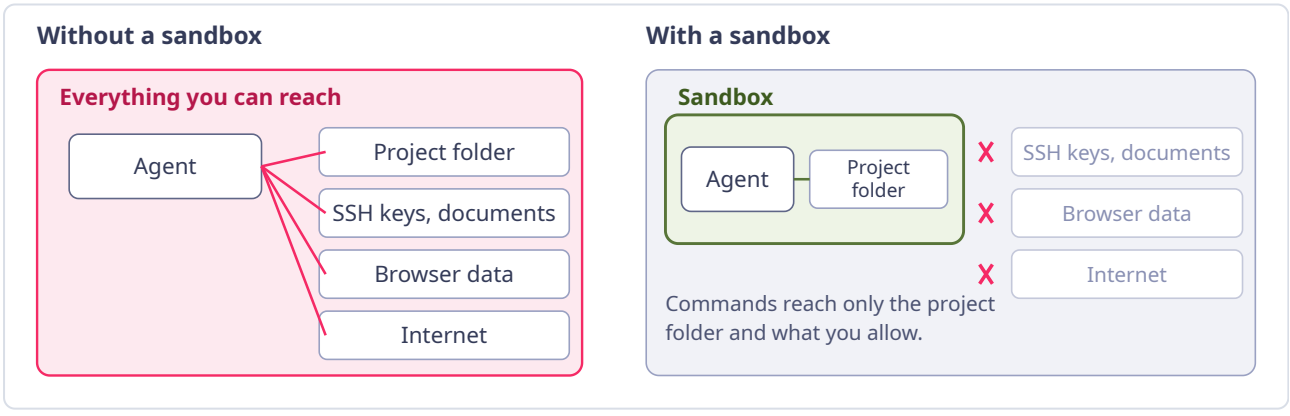


Abbildung 15 · Was eine Sandbox ändert: Befehle erreichen nur den Projektordner und was du erlaubst.

Sandbox	Ein abgeschirmter Bereich, in dem die Befehle des Agenten laufen. Darin kann ein Befehl nur den Projektordner berühren (und was du erlaubst) — nicht den Rest deines persönlichen Ordners, deine SSH-Schlüssel oder Browserdaten und oft nicht das Internet. Ohne Sandbox kann ein Befehl des Agenten alles, was du kannst.
Freigabe (zuerst fragen)	Bevor der Agent eine Datei bearbeitet oder einen Befehl ausführt, zeigt er, was er tun will, und wartet auf Ja oder Nein. Je weniger er ohne Frage darf, desto mehr Kontrolle behältst du — und desto öfter wirst du gefragt.
Erlauben- und Verbieten-Regeln	Listen von Befehlen oder Werkzeugen, die der Agent ohne Frage nutzen darf (erlauben) oder nie (verbieten). Verbieten gewinnt gegen Erlauben. Eine Regel, die jeden Shell-Befehl erlaubt, schaltet die Freigabe für die Shell aus.
Vertrauenswürdiger Ordner	Ein Ordner, in dem der Agent laut dir arbeiten darf. Dort nutzt er auch die eigenen Anweisungen und Einstellungen des Ordners, die von jemand anderem stammen können, wenn du ein Projekt klonst.
Netzwerkzugriff	Ob Befehle das Internet erreichen dürfen. Code, der deine Dateien liest und Daten verschicken kann, könnte sie preisgeben; auch das Installieren von Paketen braucht das Netzwerk.
Umgehung („YOLO“)	Alles ohne Frage und ohne Grenzen. Nur für eine Wegwerf-VM oder einen Container.
Automatische Prüfung	Ein zweites KI-Modell lässt harmlose Aktionen durch und fragt beim Rest nach.
Prompt Injection	Anweisungen, versteckt in einer Webseite, Datei oder Werkzeugantwort, die den Agenten zu etwas bringen wollen, worum du nicht gebeten hast. Zuerst fragen, eine Sandbox und kein Netzwerk stoppen sie.
Geheimnisse verbergen	Text, der wie ein Passwort, Token oder Schlüssel aussieht, wird maskiert, bevor das Modell ihn sieht oder er in einem Log landet.
Werkzeuge	Was der Agent überhaupt kann: lesen, schreiben, Befehle ausführen, im Web suchen. Ein Werkzeug, das er nicht hat, kann nicht missbraucht werden.
MCP-Server	Ein kleines Programm, das dem Agenten zusätzliche Werkzeuge gibt. Es läuft als du, mit deinen Rechten.
127.0.0.1 (localhost)	Eine Adresse, die nur Programme auf diesem PC erreichen. Ein Server auf `0.0.0.0` ist von jedem Gerät in deinem Netzwerk erreichbar.

KAPITEL 17

Add-ons

Alles, was Maprisma über einen Agenten oder einen LLM-Anbieter versteht, ist ein **Adapter**: wie er gefunden und installiert wird, seine Sitzungen, Sicherheitseinstellungen, MCP-Server und Skills. In Maprisma ist jeder Adapter ein **Add-on**: Die Pakete bringen alle mit, signiert und ausgeschaltet, und du schaltest die ein, die du nutzt. Add-ons bringen auch neue Agenten, neue Anbieter und Funktionen über die Zahlen hinaus, etwa **Insights** (Sitzungsthemen und eine Zusammenfassung von Claude). Das Plugin AI Agents Map hat die Adapter eingebaut und keine Add-ons.

17.1 Maprisma einrichten

Ein neues Maprisma öffnet sich auf seiner **Willkommensseite**. Jeder Agent und jedes lokale LLM, das Maprisma kennt, ist eine Karte mit eigenem Symbol, in drei Gruppen: **KI-Agenten**, **Lokale LLMs** und **Online-Anbieter** (die Namen von Anthropic, OpenAI und den anderen auf der Karte). Jede Karte sagt, ob das Programm hier ist:

- **Installiert** (grün) — so gefunden, wie die Karte es findet; die Karte ist schon angehakt;
- **Früher benutzt** — jetzt nicht installiert, aber seine Einstellungen sind noch da;
- **Nicht installiert** — hake es an, um es danach über seine Karte zu installieren.

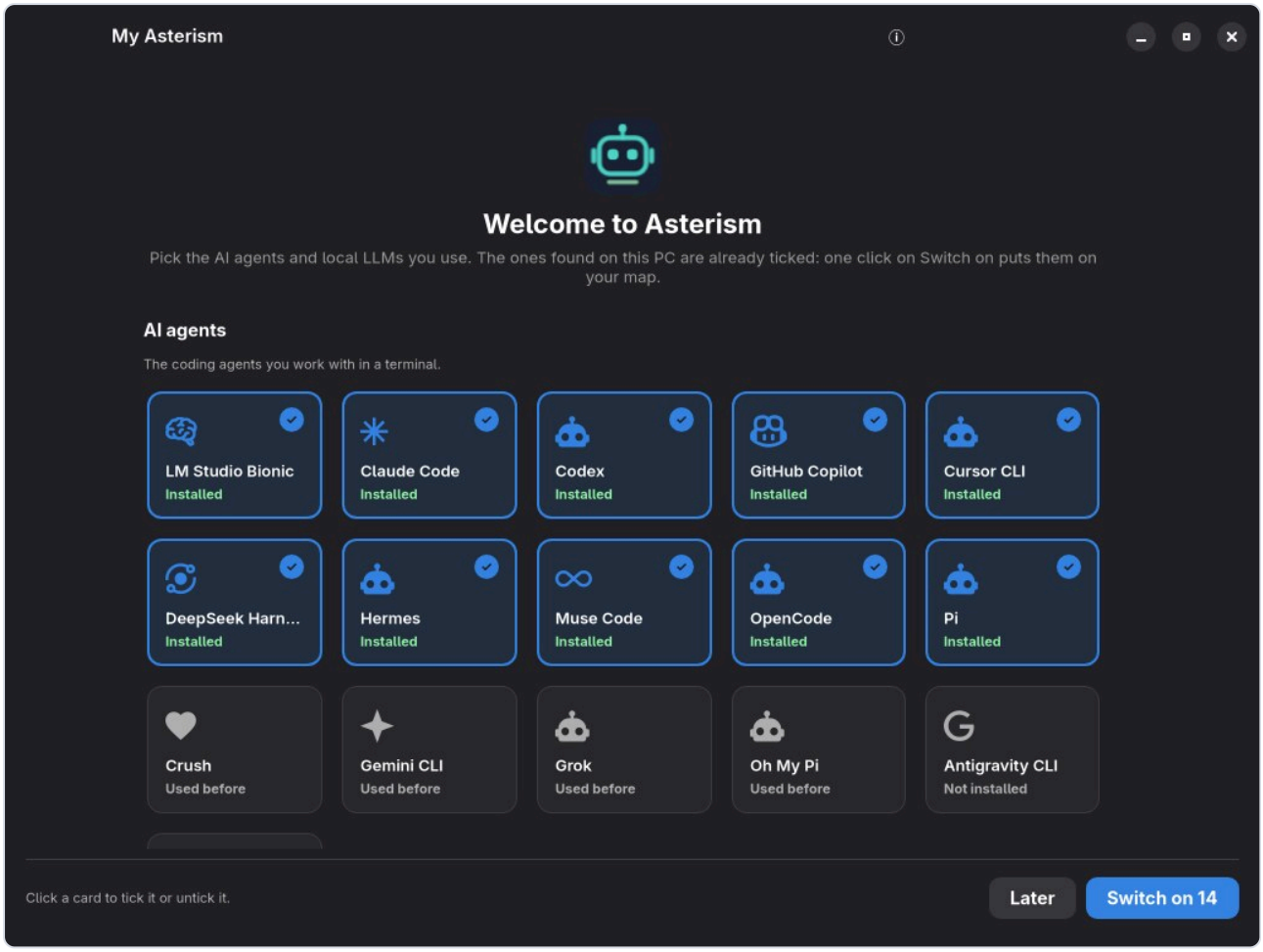


Abbildung 16 · Die Willkommensseite: gefundene Programme stehen zuerst und sind angehakt.

Klicke eine Karte an (oder drücke darauf `Leertaste`), um sie an- oder abzuhaken; eine angehakte Karte hat einen blauen Rahmen und ein Häkchen. **Einschalten** (mit der Zahl der angehakten Karten) lädt sie und zeigt die Karte; hast du Programme angehakt, die nicht installiert sind, öffnet sich **Setup**: ⋯ auf ihren Karten › **Installieren…**. **Später** startet mit einer leeren Karte. Solange nichts eingeschaltet ist, öffnet sich Maprisma bei jedem Start wieder auf der Willkommensseite. Danach sind dieselben Karten im **Setup** (dem Knopf in der Kopfzeile), wo Agenten und Anbieter jederzeit ein- und ausgeschaltet werden.

17.2 Die Add-on-Seite

Öffne sie mit dem Puzzle-Knopf in der Kopfzeile, über Einstellungen › **Add-ons** oder über jeden Hinweis „Add-on verfügbar“. Sie zeigt in zuklappbaren Gruppenkarten, die offen beginnen:

- **Vorschläge für diesen PC** — ein Add-on, dessen Agent oder Anbieter hier installiert ist, aber noch nicht gezeigt wird;
- **Installierte Add-ons** — an, aus oder *nicht geladen* mit dem Grund (zum Beispiel eine geänderte Datei); Agenten und Anbieter durch eine Linie getrennt;
- **Verfügbar** — die übrigen Add-ons des Katalogs;
- **In Maprisma eingebaut** — Agenten und Anbieter, die kein installiertes Add-on mitbringt (wenn du Maprisma aus seinem Quellordner ohne Add-ons startest).

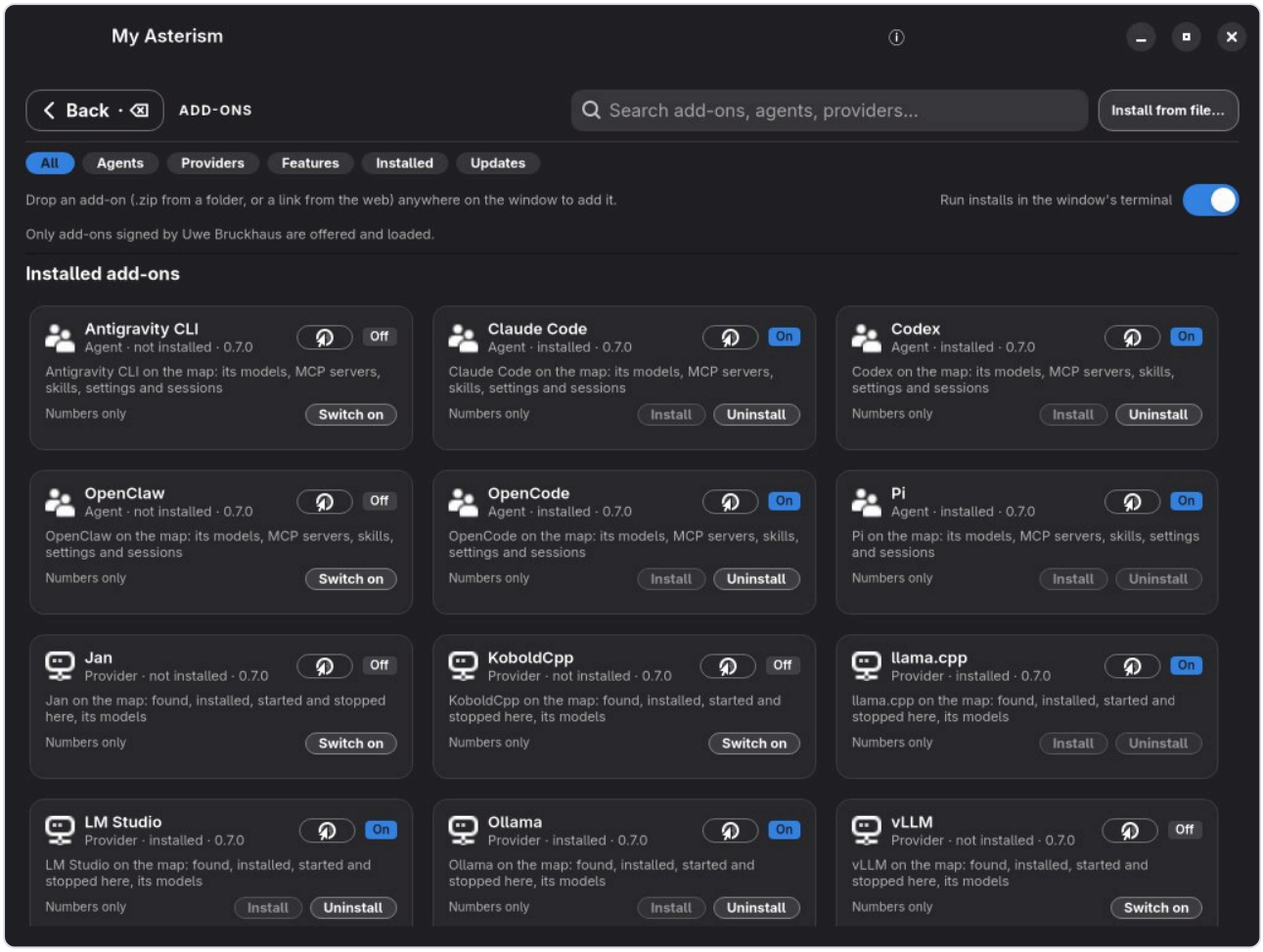


Abbildung 17 · Installierte Add-ons: an oder aus, mit Installieren und Deinstallieren für das Programm.

Die Karte eines eingeschalteten Agenten oder lokalen LLMs hat **Installieren** und **Deinstallieren** für das Programm selbst (das jeweils nicht zutreffende ist ausgegraut), und jede Karte mit einer Homepage hat einen Globus-Knopf, der sie in deinem Browser öffnet.

Die Suche findet Namen, Beschreibungen und was ein Add-on liest; die Filter darüber beschränken die Liste auf Agenten, Anbieter, Funktionen, die installierten oder die mit Update. Klicke eine Karte an für ihre Details: was es liest, was es sendet, was es darf und seine Datenschutzstufe.

17.3 Installieren, ein- und ausschalten, aktualisieren, entfernen

Installieren (oder **Add-on holen**) zeigt zuerst, was das Add-on liest, sendet und darf; erst nach deinem OK wird es heruntergeladen, geprüft und eingeschaltet. **Ausschalten** und **Einschalten** wirken sofort, ohne Neustart: Seine Agenten, Anbieter, Abschnitte und Karten erscheinen oder verschwinden überall. **Aktualisieren** ersetzt es durch die neuere Version aus dem Katalog, nachdem die neue geprüft wurde; **Entfernen** verschiebt es in den Papierkorb. **Aus Datei installieren…** nimmt ein Add-on-Zip, das du selbst heruntergeladen hast; es gelten dieselben Prüfungen.

Hat der Katalog neuere Versionen installierter Add-ons, steht auf den Karten in **Setup** *Add-on-Update*, die Seite der offenen Probleme (das rote **!** in der Kopfzeile) nennt **Add-on-Updates bereit** mit **Alle aktualisieren** und **Add-ons**, und die Add-ons-Seite zeigt eine Leiste mit **Alle aktualisieren**. Das ⋯-Menü der Karte (oder ein Rechtsklick auf sie) hat außerdem **Add-on auf** *Version* **aktualisieren**, für dieses eine Add-on. Alle aktualisieren bringt eines nach dem anderen auf den neuen Stand, mit

denselben Prüfungen, und sagt in einer Meldung, welche aktualisiert wurden. Es läuft immer nur eine Installation oder Aktualisierung; eine weitere, die währenddessen gestartet wird, bittet, es gleich noch einmal zu versuchen.

> **NUR VOM AUTOR.** Maprisma lädt nur Add-ons, die von Uwe Bruckhaus signiert sind (Schlüssel-Fingerabdruck `SHA256:e5cC634Ke3Our5LAeSc88qs22NKZEHTncBW5Ig3amWQ`). Bevor irgendetwas eines Add-ons läuft, wird die Signatur mit OpenSSHs `ssh-keygen` geprüft, jede Datei mit ihrer SHA-256-Prüfsumme verglichen, und keine Datei darf hinzugekommen sein. Ein Download muss außerdem zur SHA-256-Prüfsumme im signierten Katalog passen.

17.4 Datenschutzstufen

Stufe	Was das Add-on lesen oder senden darf
Nur Zahlen	Anzahlen, Zeiten, Tokens, geänderte Zeilen, Kosten — was Maprisma ohne Add-ons zeigt.
Sitzungstitel	Die Titel, die ein Agent seinen Sitzungen selbst gegeben hat (nie die Gespräche).
Titel und eine Zusammenfassung von Claude	Diese Titel und Zahlen, an dein eigenes Claude Code gesendet — nur auf Klick, nach einer Vorschau dessen, was genau gesendet wird.

Ein Add-on kann nie mehr tun, als seine Stufe erlaubt. Einen Agenten oder einen Anbieter hinzufügen oder ein Programm installieren sind eigene Berechtigungen, die die Karte nennt.

17.5 Arten von Add-ons

Art	Was es mitbringt
Agent	Einen Coding-Agenten: wo er seine Einstellungen, Sitzungen, MCP-Server und Skills aufbewahrt, wie er installiert wird und wie er sich anmeldet. Alles andere baut Maprisma daraus — die Karte im Setup, sein Feld auf der Karte, Nutzung und Tokens, Projekte, Zustand und Sicherheit.
Anbieter	Ein lokales LLM-Programm oder die Online-Anbieter: wie es gefunden und installiert wird und wie Agenten es erreichen. Maprisma baut seine Karte, sein Feld auf der Karte und seine Detailseite — einschließlich **Nutzung und Tokens**: was deine Agenten darüber geschickt haben, aus ihren eigenen Sitzungszahlen.
Erweiterung	Weitere Angaben zu einem Agenten oder Anbieter, den ein anderes Add-on mitbringt, ohne ihn zu ersetzen — zum Beispiel sein Abo und seine Limits oder eine Karte auf seiner Detailseite. Sie füllt nur, was das Add-on des Agenten leer lässt; die Seite des Add-ons sagt, was es ergänzt und was davon nicht genutzt wird.
Funktion	Etwas über Agenten hinweg, etwa Insights (die Karte Themen).

Der Abschnitt eines Add-ons auf der Detailseite eines Agenten oder lokalen LLMs kommt in die Karte, um die es geht: Nutzung und Tokens, Zustand, Themen (nur bei Agenten) oder eine eigene Karte mit seinem Namen.

Ein Agenten-Add-on beschreibt seinen Agenten als Angaben (eine Datei namens `agent.toml`); die meisten bringen gar keinen Code mit, und die Seite des Add-ons sagt das: *Bringt keinen Code mit* oder, für die wenigen Agenten, die etwas brauchen (etwa ihre eigene Sitzungsdatenbank), *Angaben zum Agenten und etwas Code für das, was Angaben nicht ausdrücken können*.

17.6 Ein neuer Agent auf deinem Computer

Installierst du einen Agenten, den Maprisma noch nicht kennt, sieht es im Katalog nach (Einstellungen › **Online nach Add-ons suchen**, standardmäßig an: Die Liste wird alle paar Stunden von GitHub gelesen, gesendet wird nur die Anfrage). Kennt ein Add-on diesen Agenten, sagt das **Updates und Vorschläge** (das **!** der Kopfzeile) (*„Kilo Code ist auf diesem PC, aber nicht auf der Karte“*), und eine Karte erscheint unter **Vorschläge für diesen PC**; ein Klick installiert es. Gibt es noch kein Add-on, sagen das die Details des Agenten, mit einem Link, um eines zu erbitten.

17.7 Ein Add-on erstellen

Ein neues lokales LLM oder ein neuer Agent braucht nichts als ein Add-on: Maprisma baut seine Karte, seinen Kasten und seine Linien auf der Karte, seine Details, die Modelle-Seite und die Sicherheitsseite aus dem, was das Add-on darüber sagt. Wenn du ein wenig programmieren kannst, fang mit den Vorlagen im Maprisma-Repository an (`templates/`, erklärt in `docs/addons.md`): eine Datei für ein lokales LLM, eine Datei mit Fakten für einen Agenten. Schick das Add-on an `github.com/uBruckhaus/asterism-addons`; ist es geprüft und signiert, erscheint es für alle unter **Verfügbar**.

KAPITEL 18

Datenschutz und Sicherheit

- **Keine Administratorrechte für die App.** Sie läuft nie als root und sieht nie ein Passwort. Einen Agenten, der als Systempaket kommt (etwa Hermes oder OpenClaw), installiert die Paketverwaltung des Systems mit `sudo` in einem Terminal, das du siehst, nachdem dir der Befehl gezeigt wurde.
- **Keine Zugangsdaten gelesen.** Sie liest die Einstellungen der Agenten, aber nie ihre Zugangsdateien. Ob ein Agent angemeldet ist, wird daran erkannt, ob eine solche Datei existiert. Werte von Schlüsseln und Headern werden nie gezeigt.
- **Keine Gespräche gelesen.** Nutzungszahlen werden nur als Zahlen und Zeiten gelesen. Die einzige Ausnahme ist das Mitlesen einer Instanz *Im Hintergrund* (Terminals…), auf deinen Klick und nur für diese Instanz; nichts davon wird gespeichert.
- **Netzwerk nur auf Wunsch** — Modelle, MCP-Server oder Skills suchen, installieren, einen Server testen —, dazu alle paar Stunden eine Update-Prüfung.
- **Geprüfte Downloads.** Programme werden nur installiert, wenn der Download zur Prüfsumme passt, die sein Release veröffentlicht.
- **Vorschau und Rückgängig.** Jede Änderung an den Einstellungen eines anderen Programms wird vorab gezeigt, gesichert und lässt sich rückgängig machen. Einstellungen, die Schlüssel enthalten können, werden nur für dich lesbar geschrieben.
- **Begrenzt.** Jedes Programm läuft ohne Shell, und jede Antwort wird mit Größen- und Zeitgrenze gelesen.
- **KI-Funktionen nur, wenn du sie wählst.** Das Nachschlagen von Smart Check läuft auf Klick, die täglichen KI-Prüfungen nur, wenn sie eingeschaltet sind. Der Agent bekommt eine kurze Zusammenfassung (Namen, Versionen, Modelle), nie Schlüssel oder Einstellungswerte, übergeben so, dass andere Benutzer des Computers sie nicht lesen können; seine Antworten werden geprüft, bevor sie gezeigt werden, und nie ausgeführt. Aufgaben laufen in einem Terminal, das du siehst, und der Agent fragt vor jedem Schritt. Du nutzt sie auf eigene Gefahr.

Die README in jedem Repository listet vollständig auf, was gelesen, gestartet und geschrieben wird.

KAPITEL 19

Problemlösung

Problem	Was zu tun ist
Die Karte ist gedrängt oder abgeschnitten.	Das Fenster ist schmaler als etwa 1300 Pixel. Mach das Fenster breiter, oder zoome heraus (`Strg`+`-`).
Ein Feld, das ich erwarte, ist nicht auf der Karte.	Die Karte zeigt nur, was eingerichtet ist: Sieh im **Setup** nach (ein ausgeschaltetes Add-on, ein MCP-Server, den kein Agent nutzt) und bei den **Zeigen**-Schaltern.
Hier starten fehlt oder tut nichts.	Installiere VTE für GTK 4 (`vte4`), oder nimm **Im Terminal starten**.
Eine Linie ist rot gepunktet.	Klicke sie an für Hinweise, behebe, was sie sagen, dann **Erneut testen**. Siehe Wenn eine Verbindung fehlschlägt.
Das lokale LLM startet nicht mit dem Agenten.	Prüfe Einstellungen › **Lokale LLMs mit ihren Agenten starten** und dass der Agent auf der Karte verbunden ist. `systemctl --user status ai-agents-map-providers` zeigt den Wächter.
Das farbige Licht von einem lokalen LLM zu seinem Modell erscheint nicht.	Es erscheint nur, während ein Programm, das nicht auf der Karte ist, mit diesem LLM erzeugt, und verblasst etwa zehn Sekunden nach seiner letzten Anfrage. Prüfe, ob die Karte sichtbar ist (ein breites Fenster), ob das Programm wirklich etwas sendet und ob Einstellungen › **Karten-Animationen** auf **Agenten und andere Programme** steht.
Der Agent antwortet schlecht oder bricht mit einem lokalen Modell ab.	Der Kontext ist vielleicht zu klein. Öffne das Modell unter **Modelle… › Installiert** und nutze **Kontext setzen** mit dem besten Wert für deine Karte.
Ein Download ist stehengeblieben.	Nach einem Verbindungsabbruch setzt er sich selbst fort. Nach einem Neustart nimm **Fortsetzen** im Streifen oben.
Ich will eine frühere Änderung rückgängig machen.	Sicherungen liegen in `~/.local/state/ai-agents-map/backups/` .

ANHANG

Referenz

A.1 Tastenkürzel

`F1`	Hilfe
`Strg`+`,`	Einstellungen
`F5`	Aktualisieren (die Karte und Setup)
`Strg`+`+` / `Strg`+`-` / `Strg`+`0`	Die Karte vergrößern, verkleinern, zurück auf 100 %
`Strg`+Scrollen	Die Karte zoomen
`Umschalt`+Klick	Zur Auswahl hinzufügen oder daraus entfernen
`Esc`	Auswahl aufheben; von einer Seite zurück
`Rücktaste`	Von einer Seite zurück

A.2 Bedienung per Tastatur in der Flutter-Vorschau

Die Kürzel oben gelten auch in der Flutter-Vorschau: `F1`, `Strg`+`,`, `F5`, `Strg`+`+` / `Strg`+`-` / `Strg`+`0` für die sichtbare Karte und `Esc` oder `Rücktaste`, um von einer Seite zurückzugehen. In einem Textfeld löscht die `Rücktaste`, ein Dialog schließt sich mit `Esc`, und ein Terminal behält beide Tasten für seinen Agenten.

`Tab` geht durch alles auf dem Bildschirm, `Enter` oder `Leertaste` benutzt es. Auf der Karte öffnet `Enter` auf einem Kasten diesen, und sein Knopf ⋮ öffnet seine Aktionen. Im Karten-Editor hat jedes Ziehen einen Weg per Tastatur: **Aktionen** › **Verbinden** eines Kastens, dann mit `Tab` zum Zielkasten und `Enter` (`Esc` bricht ab); **Verbindung verschieben**, **Nach oben** und **Nach unten** stehen im selben Menü. Die App wird auf beschriftete Bedienelemente, Ziele von mindestens 24 × 24 Pixeln und einen Textkontrast von mindestens 4,5:1 geprüft, hell und dunkel.

A.3 Dateien und Ordner

Pfad	Inhalt
`~/.config/maprisma/` MAPRISMA `~/.config/omarchy/ai-agents-map/` OMARCHY	Einstellungen, die Liste der MCP-Server, nachgeschlagene Agenten, fehlgeschlagene Verbindungen.
`~/.local/state/ai-agents-map/backups/`	Sicherungen jeder Datei, die die App geändert hat.
`~/.local/state/ai-agents-map/jobs.json`	Unterbrochene Downloads und Installationen.

Pfad	Inhalt
`~/.local/state/ai-agents-map/reorganize.json`	Das Journal des letzten Aufräumens, für Rückgängig.
`~/.cache/ai-agents-map/`	Suchergebnisse, Werkzeuglisten, Update-Prüfungen, der letzte Smart Check (`smart-check.json`) und die MCP-Updates der KI-Prüfungen (`mcp-updates.json`).
`~/.local/state/ai-agents-map/ai-help/`	Das Protokoll: eine Datei pro Lauf der KI-Hilfe (die letzten 200).
`~/.local/state/omarchy-plugin-ai/runs.json`	Das Nachschlagen und die Aufgaben dieser App und anderer Plugins, mit ihren Tokens: die Statistik von Smart Check.
`~/.config/systemd/user/ai-agents-map-providers.service`	Der Wächter, der lokale LLMs mit ihren Agenten startet.

A.4 Glossar

Freigabe Ein Agent fragt, bevor er eine Datei ändert oder einen Befehl ausführt (siehe Die Begriffe erklärt).

Kontext Wie viele Tokens Text ein Modell auf einmal im Blick behalten kann.

GGUF Das Dateiformat für Modelle, das llama.cpp, LM Studio, Ollama und KoboldCpp nutzen.

Prompt Injection Versteckte Anweisungen in einer Seite, Datei oder Werkzeugantwort, die einen Agenten lenken wollen.

Quantisierung Ein Modell mit weniger Bits pro Gewicht speichern (zum Beispiel *Q4_K_M*, etwa 4,8 Bit), damit es kleiner ist, bei geringem Qualitätsverlust.

Mixture of Experts Ein Modell, bei dem für jedes Token nur ein Teil der Parameter arbeitet (*35B-A3B*: 35 Milliarden insgesamt, 3 Milliarden aktiv), was es für seine Größe schnell macht.

Sandbox Ein abgeschirmter Bereich, in dem die Befehle eines Agenten laufen und nur den Projektordner und das Erlaubte erreichen.

Token Ein Stück Text, ungefähr drei Viertel eines Wortes, das Modelle lesen und schreiben.

Benutzerdienst Ein Hintergrundprogramm, das systemd nur für deinen Benutzer startet, ohne Administratorrechte.